

---

# **Security Review Report**

## **NM-1010 - Suno**

---



**NETHERMIND**  
**SECURITY**

(September 14, 2026)

# Contents

|          |                                                                                                                              |           |
|----------|------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Executive Summary</b>                                                                                                     | <b>2</b>  |
| <b>2</b> | <b>Audited Files</b>                                                                                                         | <b>3</b>  |
| <b>3</b> | <b>Summary of Issues</b>                                                                                                     | <b>3</b>  |
| <b>4</b> | <b>System Overview</b>                                                                                                       | <b>4</b>  |
| 4.1      | Tokenized Assets                                                                                                             | 4         |
| 4.2      | The Reserve Architecture                                                                                                     | 4         |
| 4.3      | Energy Attestation and State Updates                                                                                         | 4         |
| 4.4      | Yield Distribution Flow                                                                                                      | 4         |
| 4.5      | Assumptions and Trust Model                                                                                                  | 4         |
| <b>5</b> | <b>Risk Rating Methodology</b>                                                                                               | <b>6</b>  |
| <b>6</b> | <b>Issues</b>                                                                                                                | <b>7</b>  |
| 6.1      | [High] While the collateral ratio is under par, transactions that raise it can be sandwiched for risk-free profit            | 7         |
| 6.2      | [Medium] Hardcoded USDC value in the YieldController contract dilutes protocol collateralization                             | 11        |
| 6.3      | [Medium] Inconsistent valuation of custody USDC moves the collateral ratio spuriously, and the move can be sandwiched        | 12        |
| 6.4      | [Medium] The risk posture sizing rule understates the collateral ratio needed to absorb the design shock                     | 14        |
| 6.5      | [Low] Disabled deposit assets break the protocol's cash valuation and operations                                             | 16        |
| 6.6      | [Low] Dynamic daily redemption limit restricts withdrawals more than intended                                                | 17        |
| 6.7      | [Low] The de-peg band admits one basis point more than the redemption fee, making a deposit and redeem round trip profitable | 18        |
| 6.8      | [Low] The energy issuance path in the SunoLogicReserve contract does not enforce the solar concentration ceiling             | 20        |
| 6.9      | [Low] Users can only have their project tokens swapped once causing subsequent tokens to be stuck                            | 22        |
| 6.10     | [Info] Atomic acquisitions and disposals incorrectly refresh the aggregate energy valuation timestamp                        | 23        |
| 6.11     | [Info] Incorrect event emission in the swap function causes a permanent denial of service via the reconciliation monitor     | 24        |
| 6.12     | [Info] The BLOCKER_ROLE can freeze all uWatt issuance and redemption, which pausing is supposed to gate                      | 25        |
| <b>7</b> | <b>Documentation Evaluation</b>                                                                                              | <b>27</b> |
| <b>8</b> | <b>Test Suite Evaluation</b>                                                                                                 | <b>28</b> |
| 8.1      | Tests Output                                                                                                                 | 28        |
| <b>9</b> | <b>About Nethermind</b>                                                                                                      | <b>33</b> |

# 1 Executive Summary

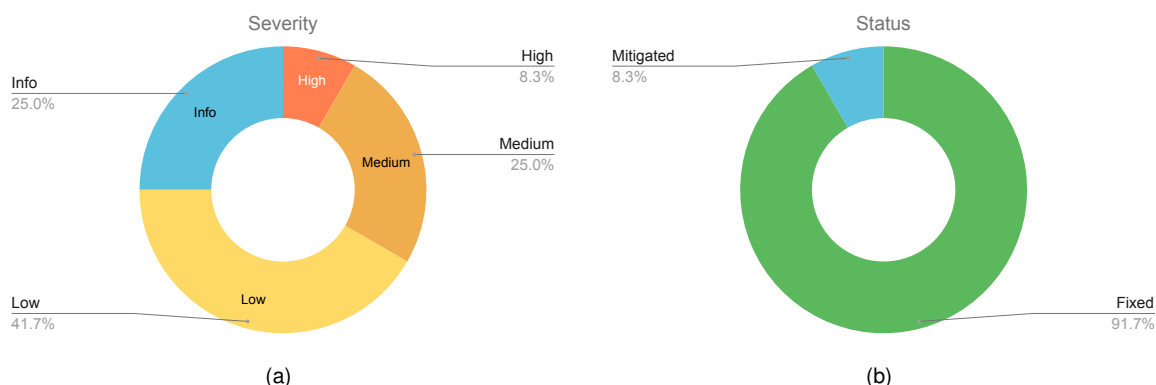
This document presents the results of the security review conducted by [Nethermind Security](#) for [Suno Protocol's](#) smart contracts. Suno Protocol's team has developed a modular, collateral-backed issuance protocol that tokenizes real-world solar energy infrastructure through two assets: **pWatt**, a project-specific instrument that finances the development of new solar projects, and **uWatt**, a token that provides exposure to Suno's entire portfolio of operational, revenue generating projects. Issuance and redemption are handled by the **uWattIssuer** contract.

The protocol implements a yield distribution mechanism through the **YieldController** contract, which receives energy revenue in USDC and then distributes it. The protocol is built with upgradeability in mind using the UUPS proxy pattern, with the project tokens sharing a single implementation behind a governance-owned beacon, allowing for future logic enhancements while maintaining an access control system across issuance, energy accounting, distribution, and cross-chain components.

**The audit comprises 2472 lines of Solidity code. The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

**Along this document, we report** twelve points of attention, where one is classified as High, three are classified as Medium, five are classified as Low and three are classified as Informational.

**This document is organized as follows.** Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.



**Fig. 1: Distribution of issues: Critical (0), High (1), Medium (3), Low (5), Undetermined (0), Informational (3), Best Practices (0). Distribution of status: Fixed (11), Acknowledged (0), Mitigated (1), Unresolved (0)**

## Summary of the Audit

|                                 |                         |
|---------------------------------|-------------------------|
| <b>Audit Type</b>               | Security Review         |
| <b>Initial Report</b>           | August 21, 2026         |
| <b>Final Report</b>             | September 14, 2026      |
| <b>Initial Commit</b>           | <a href="#">c96c981</a> |
| <b>Final Commit</b>             | <a href="#">e1555fb</a> |
| <b>Documentation Assessment</b> | Medium                  |
| <b>Test Suite Assessment</b>    | Medium                  |

## 2 Audited Files

|    | File                                                              | LoC         | Comments    | Ratio        | Blank      | Total       |
|----|-------------------------------------------------------------------|-------------|-------------|--------------|------------|-------------|
| 1  | <a href="#">contracts/UWattPriceOracle.sol</a>                    | 52          | 38          | 73.1%        | 14         | 104         |
| 2  | <a href="#">contracts/EnergyAssetsReporter.sol</a>                | 97          | 67          | 69.1%        | 19         | 183         |
| 3  | <a href="#">contracts/CUWattPriceOracle.sol</a>                   | 22          | 43          | 195.5%       | 6          | 71          |
| 4  | <a href="#">contracts/UWattIssuer.sol</a>                         | 583         | 335         | 57.5%        | 118        | 1036        |
| 5  | <a href="#">contracts/ERC20UWattUpgradeable.sol</a>               | 162         | 103         | 63.6%        | 51         | 316         |
| 6  | <a href="#">contracts/ChainlinkStableOracle.sol</a>               | 33          | 23          | 69.7%        | 12         | 68          |
| 7  | <a href="#">contracts/adapters/SkySUSDSAdapter.sol</a>            | 115         | 59          | 51.3%        | 33         | 207         |
| 8  | <a href="#">contracts/crosschain/SunoLayerZeroAdapter.sol</a>     | 102         | 19          | 18.6%        | 22         | 143         |
| 9  | <a href="#">contracts/crosschain/SunoLockbox.sol</a>              | 54          | 22          | 40.7%        | 14         | 90          |
| 10 | <a href="#">contracts/crosschain/SunoPriceOracleL2.sol</a>        | 151         | 117         | 77.5%        | 18         | 286         |
| 11 | <a href="#">contracts/crosschain/XCUWatt.sol</a>                  | 232         | 77          | 33.2%        | 40         | 349         |
| 12 | <a href="#">contracts/governance/TimelockImport.sol</a>           | 2           | 3           | 150.0%       | 1          | 6           |
| 13 | <a href="#">contracts/interfaces/IEnergyAssetsReporter.sol</a>    | 8           | 23          | 287.5%       | 5          | 36          |
| 14 | <a href="#">contracts/interfaces/IUWattIssuer.sol</a>             | 76          | 54          | 71.1%        | 28         | 158         |
| 15 | <a href="#">contracts/interfaces/IBondAdapter.sol</a>             | 19          | 36          | 189.5%       | 12         | 67          |
| 16 | <a href="#">contracts/interfaces/IStableOracle.sol</a>            | 4           | 10          | 250.0%       | 1          | 15          |
| 17 | <a href="#">contracts/interfaces/IERC20UWatt.sol</a>              | 7           | 11          | 157.1%       | 4          | 22          |
| 18 | <a href="#">contracts/interfaces/IUWattPriceOracle.sol</a>        | 4           | 14          | 350.0%       | 1          | 19          |
| 19 | <a href="#">contracts/interfaces/external/IAggregatorV3.sol</a>   | 15          | 2           | 13.3%        | 3          | 20          |
| 20 | <a href="#">contracts/interfaces/external/ISkyPsm.sol</a>         | 7           | 15          | 214.3%       | 4          | 26          |
| 21 | <a href="#">contracts/interfaces/external/IERC4626Minimal.sol</a> | 7           | 10          | 142.9%       | 4          | 21          |
| 22 | <a href="#">contracts/core/SunoProjects.sol</a>                   | 132         | 60          | 45.5%        | 28         | 220         |
| 23 | <a href="#">contracts/core/SunoLogicReserve.sol</a>               | 244         | 120         | 49.2%        | 48         | 412         |
| 24 | <a href="#">contracts/core/YieldController.sol</a>                | 111         | 44          | 39.6%        | 36         | 191         |
| 25 | <a href="#">contracts/core/SunoRegistry.sol</a>                   | 54          | 24          | 44.4%        | 14         | 92          |
| 26 | <a href="#">contracts/core/SunoStakingVault.sol</a>               | 89          | 70          | 78.7%        | 24         | 183         |
| 27 | <a href="#">contracts/core/SunoProjectToken.sol</a>               | 85          | 55          | 64.7%        | 27         | 167         |
| 28 | <a href="#">contracts/core/SunoProjectTokenBeacon.sol</a>         | 5           | 12          | 240.0%       | 2          | 19          |
|    | <b>Total</b>                                                      | <b>2472</b> | <b>1466</b> | <b>59.3%</b> | <b>589</b> | <b>4527</b> |

## 3 Summary of Issues

|    | Finding                                                                                                                | Severity | Update    |
|----|------------------------------------------------------------------------------------------------------------------------|----------|-----------|
| 1  | While the collateral ratio is under par, transactions that raise it can be sandwiched for risk-free profit             | High     | Fixed     |
| 2  | Hardcoded USDC value in the YieldController contract dilutes protocol collateralization                                | Medium   | Fixed     |
| 3  | Inconsistent valuation of custody USDC moves the collateral ratio spuriously, and the move can be sandwiched           | Medium   | Fixed     |
| 4  | The risk posture sizing rule understates the collateral ratio needed to absorb the design shock                        | Medium   | Fixed     |
| 5  | Disabled deposit assets break the protocol's cash valuation and operations                                             | Low      | Fixed     |
| 6  | Dynamic daily redemption limit restricts withdrawals more than intended                                                | Low      | Mitigated |
| 7  | The de-peg band admits one basis point more than the redemption fee, making a deposit and redeem round trip profitable | Low      | Fixed     |
| 8  | The energy issuance path in the SunoLogicReserve contract does not enforce the solar concentration ceiling             | Low      | Fixed     |
| 9  | Users can only have their project tokens swapped once causing subsequent tokens to be stuck                            | Low      | Fixed     |
| 10 | Atomic acquisitions and disposals incorrectly refresh the aggregate energy valuation timestamp                         | Info     | Fixed     |
| 11 | Incorrect event emission in the swap function causes a permanent denial of service via the reconciliation monitor      | Info     | Fixed     |
| 12 | The BLOCKER_ROLE can freeze all uWatt issuance and redemption, which pausing is supposed to gate                       | Info     | Fixed     |

## 4 System Overview

Suno Protocol is a DeFi platform designed to tokenize real-world solar energy infrastructure. By connecting physical clean energy production with blockchain infrastructure, Suno enables users to finance new solar projects and earn yield from operational ones. The protocol relies on a system of smart contracts to manage token issuance, collateral accounting, and yield accrual. Through its dual-token model, transparent reserve accounting, and on-chain NAV oracles, Suno provides exposure to a portfolio of clean energy assets whose yield derives from real-world energy sales.

### 4.1 Tokenized Assets

Suno's ecosystem uses a two-token model that separates the financing of new projects from exposure to the operational portfolio.

- **pWatt:** Project-specific instruments used to raise capital for a single, distinct solar project during its funding and development phase. Each pWatt is tied exclusively to the development and completion of that specific project. Its lifecycle runs from funding through physical deployment to operation, at which point pWatt holders deposit their tokens into The Reserve and receive newly issued uWatt in return.
- **uWatt:** A liquid, fungible, yield-bearing token backed by the protocol's diversified portfolio of operational projects complemented by a liquid sleeve of bonds and stablecoins, known as The Reserve. It is designed for holding, trading, and gaining exposure to the full set of operational solar projects rather than to any single one. Because uWatt yield is derived from the aggregate performance of many projects and the liquid sleeve absorbs part of the volatility of the solar valuation, its exposure to single-project variance is reduced.

### 4.2 The Reserve Architecture

The SunoLogicReserve contract functions as the entry point through which operational solar projects are aggregated into The Reserve, the pool of tokenized solar projects, bonds and cash that backs uWatt.

- **Portfolio Aggregation:** As individual solar projects go live, their pWatt tokens are deposited into the SunoLogicReserve contract and the value of the project is booked into the aggregate energy valuation. This aggregation reduces the risk of any single project underperforming, standardizing the yield for all uWatt holders.
- **Asset Conversion Flow:** The SunoLogicReserve contract provides the mechanism through which projects enter the diversified portfolio. Once a project is operational, the swap operator deposits its holders' pWatt tokens into the SunoLogicReserve contract. In exchange, the system values the project at the auditor-set price and mints a corresponding amount of newly issued uWatt tokens to each holder, absorbing the individual project into the broader diversified portfolio.
- **Valuation and Metrics:** The smart contracts track the underlying assets within The Reserve through the EnergyAssetsReporter contract, where the valuation of the active projects is reported onchain by authorized roles within a per-update band and subject to a staleness check. The collateral ratio of uWatt is derived from this valuation together with the bonds and cash held in custody, and uWatt is issued and redeemed at min(\$1, C).

### 4.3 Energy Attestation and State Updates

To link off-chain solar generation with on-chain financial mechanics, Suno relies on a data ingestion system built utilizing an Energy Attestation Service (EAS).

- **Off-chain:** The energy production of the physical solar installations is monitored.
- **On-chain Attestation:** Authorized entities utilize the protocol's Energy Attestation contract to submit these values on-chain. The attestations act as proofs of the energy generated over a specific time epoch.

### 4.4 Yield Distribution Flow

The protocol features a revenue sharing mechanism that distributes yield to uWatt holders.

- **Revenue Injection:** Revenue generated from the physical sale of electricity is converted into on-chain assets (typically stablecoins) and deposited into the protocol's custody account through the YieldController contract.
- **Yield Calculation:** The YieldController contract splits each incoming payment according to the protocol's collateral ratio. A portion is retained in The Reserve and the remainder is minted as uWatt to the SunoStakingVault, where it vests linearly to stakers.

### 4.5 Assumptions and Trust Model

The review was carried out under the following assumptions about the deployment environment and the way the protocol is operated.

- **Canonical cash asset:** USDC remains an enabled deposit asset for the lifetime of the deployment, transferable for the custody account and backed by a maintained, fresh price feed.
- **Trusted roles, data and integrations:** The administrative, treasury, oracle, attestation and pause roles act in the protocol's interest and, beyond being non-malicious, are exercised continuously. The design deliberately keeps part of the safety perimeter off-chain so that operators retain flexibility, which in turn assumes that monitoring, acknowledgement of legitimate valuation moves, and the

upkeep of risk parameters are performed diligently and in a sequence that preserves the protocol's intended invariants. Likewise, energy attestations and price feeds are assumed to reflect reality and arrive within their expected cadence, and integrated contracts such as the bond adapters, custody account and accepted token contracts to conform to their documented interfaces.

## 5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

**Likelihood** measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

**Impact** is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

|        |              | Severity Risk       |              |              |
|--------|--------------|---------------------|--------------|--------------|
| Impact | High         | Medium              | High         | Critical     |
|        | Medium       | Low                 | Medium       | High         |
|        | Low          | Info/Best Practices | Low          | Medium       |
|        | Undetermined | Undetermined        | Undetermined | Undetermined |
|        |              | Low                 | Medium       | High         |
|        |              | Likelihood          |              |              |

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

## 6 Issues

### 6.1 [High] While the collateral ratio is under par, transactions that raise it can be sandwiched for risk-free profit

**File(s):** `contracts/UWattIssuer.sol`, `contracts/UWattIssuer.sol`, `contracts/UWattIssuer.sol`, `contracts/core/YieldController.sol`

**Description:** The UWattIssuer contract runs a capped peg. Users mint uWatt through the `depositAndMint(...)` function and sell it back through the `redeemUWatt(...)` function, both priced at  $\min(\$1, C)$ , where  $C$  is the collateral ratio. Pricing both sides identically is what makes issuance value neutral: before the redemption fee, a depositor mints at the same value a redeemer would receive.

While the reserve holds at least a dollar per token the cap binds, both sides sit at par, and movements in  $C$  are invisible to the desk. Governance can also run the protocol below par: the `cFloorWad` parameter is described in the code as "the rescue floor", and lowering it under  $1e18$  deliberately opens minting while  $C < 1$  so new depositors can recapitalise an impaired reserve. In that mode the cap stops binding and  $C$  becomes a live price on both sides:

```
1 uint256 rawC = _uWattPriceStalenessChecked(); // reserveValue/supply, fail-closed
2 // @audit Below par the cap does not bind, so the mint price is the live C.
3 mintPriceWAD = rawC > WAD ? WAD : rawC; // min($1, C)
```

A limit used to exist on how far the desk's price could move between one operation and the next, which the code calls the jump band. It no longer applies, and the comment retiring it explains the reasoning:

```
1 // (b) below par, min($1,C)=C only descends via reporter moves already banded
2 // at 30 bps per report (+ governance reportUnchecked, which pairs with a
3 // pause) - manipulation is bounded at the INPUT, not at the desk;
4 // ...
5 uint256 rawPrice = _uWattPriceStalenessChecked();
6 // @audit-issue No limit is applied on how far C moved since the matching mint.
7 uint256 price = rawPrice > WAD ? WAD : rawPrice;
```

The `maxPriceJumpBps` and `lastPriceWAD` variables and the `acceptPriceJump(...)` function all still exist, and `lastPriceWAD` is still written on every redemption, but nothing reads them when pricing; the contract documents them as retained "for monitor continuity" only.

That reasoning considers only the direction in which  $C$  **descends**, and correctly observes that the reporter's 30 bps per update band keeps such moves under the 50 bps redemption fee. It does not cover the direction in which  $C$  **ascends**, which does not pass through the reporter at all. Since  $C$  is what the `reserveValue(...)` function returns over supply, and the `cashValue(...)` function is computed from custody's balance,  $C$  rises whenever cash arrives. The `distributeEnergyPayment(...)` function of the `YieldController` contract does exactly that as a routine treasury action:

```
1 IERC20(issuer.usdc()).safeTransferFrom(_msgSender(), issuer.custody(), usdcIn);
```

The resulting step in  $C$  is bounded only by the payment's size relative to the reserve, there is no band on it, and the transaction is visible in the mempool before it lands.

A treasury payment is not the only way the reserve grows. Custody cash is marked at whatever the stablecoin feed reports, and that valuation deliberately carries no de-peg band:

```
1 // Deliberately freshness only, NOT the de-peg band: reverting valuation because
2 // a stablecoin trades at $0.97 would take the desk down exactly when redemptions
3 // matter most. Overstating is the risk here; refusing to price is the mitigation.
4 // @audit Overstating raises C with no bound, which the desk no longer checks.
5 uint256 p = _stablePriceFresh(asset);
```

Accepting an overstatement is a reasonable trade for keeping the desk open during stress, and while the desk applied a band of its own that band bounded what a revaluation could be monetised for. Without one it is directly monetisable: a stablecoin printing above par revalues the whole custody position and the same bracket pays out, with no privileged call involved. How large a print is needed follows from the cash share of the reserve, since  $dC / C$ , meaning the relative change in the collateral ratio a single transaction produces, is the price move scaled by that share. At roughly 31% cash it takes about 1.6%, which is a large print for a liquid stablecoin, but the threshold falls as the liquid sleeve grows and converges on the fee itself at 100% cash, so the exposure is a function of reserve composition and the risk posture governs that.

The bond leg is the third term, and it carries no per-update band either:

```
1 usd +=
2 // @audit The bond leg carries no per-update band of any kind.
3 (IERC20(bond).balanceOf(custody) * IBondAdapter(entry.adapter).navWAD()) /
4 _bondUnit(entry);
```

So the reserve is a three term sum, and the reasoning that retired the desk band bounds only one of them. The energy term is limited by `maxChangeBps` to 30 bps per update, which stays under the fee for any reserve composition, since a move of 30 bps in one term can move `C` by at most 30 bps times that term's share. The bond and cash terms are limited by nothing in this codebase. They are quiet in the current deployment only because `sUSDS` accrues smoothly and Chainlink's USDC feed is well behaved, which are properties of external systems rather than guarantees of this one, and even for `sUSDS` a change to the Sky PSM's `tout` parameter is a step rather than an accrual.

From a deposit taken near par none of the three clears the fee alone. What they do is add to a cash inflow, so a payment that would not otherwise be worth bracketing becomes one that is. A payment moving `C` by 39 bps costs the attacker money on its own, but the same payment landing in a block that also carries a legitimate `report(...)` at exactly the 30 bps band and one routine Chainlink print at USDC's 0.25% deviation threshold moves `C` by 59 bps and turns a profit. Neither contributing transaction is anomalous, and the attacker does not need to cause either, only to wait for a block that carries them, all of which are visible in the mempool beforehand. The practical effect is on the size of victim payment required, which is what determines how often the opportunity appears: against the reserve used below it falls by roughly 38%.

An attacker can therefore bracket the payment inside a single block, minting before it and redeeming after. The anti round trip guard does not prevent this, because it compares `lastMintBlock[msg.sender]` against `block.number` for a single address and the two legs can be split across two attacker owned addresses, the same bypass the code describes elsewhere as being "sidestepped with a second account".

The profit is  $(dC / C) - \text{fee}$  on the deposited amount. Both legs settle in one block, so the attacker carries no market exposure between them. Two refinements matter. First, that form holds only while USDC reads exactly \$1: the deposit is credited at `amountIn * oraclePrice`, and the `setDepositAsset(...)` function admits a premium up to `maxDeviationBps`, itself capped at the fee, so the general form is  $(dC / C) - \text{fee} + \text{premium}$  and any admitted premium subtracts directly from the hurdle. Second, the redemption still pays  $\min(\$1, C)$ , so a payment large enough to carry `C` back over par is capped at \$1 while the mint was priced lower. Written out, the profit is  $\min(\$1, C_{\text{after}}) / C_{\text{before}} * (1 - \text{fee}) - 1$ , which splits into a payment-limited regime of  $(dC / C) - \text{fee}$  while `Cafter` stays under par, and a depth-limited regime of  $(1 - \text{fee}) / C_{\text{before}} - 1$  once the cap binds, independent of payment size. A single sandwich is therefore bounded by how far below par the mint happened, and in the capped regime it breaks even exactly at `Cbefore = 1 - fee`.

A concrete sequence, with the reserve holding 1,250,000 USD of energy assets, 1,000,000 USD of bonds and 1,000,000 USDC of custody cash against a supply of 3,332,943.635308 uWatt:

1. Governance has lowered `cFloorWad` beneath the prevailing `C`, and the reserve is impaired, so `C` is 0.975114 and minting below par is open;
2. The attacker observes a pending call to the `distributeEnergyPayment(...)` function for 34,160.749807 USDC;
3. Front running it, attacker address A calls the `depositAndMint(...)` function with 169.637383 USDC and receives 173.966718763139489331 uWatt, priced at `C = 0.975114`;
4. Address A transfers that uWatt to attacker address B in the same block, which is the step that defeats the per address guard;
5. The victim transaction lands. Custody cash becomes 1,034,330.387190 USDC, the reserve 3,284,330.387190 USD, and `C` rises to 0.985363;
6. Address B calls the `redeemUWatt(...)` function for the full balance and receives 170.563246 USDC;

That is a profit of 0.925863 USDC, or +0.546%, inside a single block: the `C` move of +1.051% exceeded the 0.50% fee, and everything above the fee is profit.

Three factors scale that result, all of them visible in the formula above. The payment's size relative to the reserve sets `dC / C`, so a larger payment pays more. Impairment depth does not enter `dC / C`, since that ratio measures the payment against the reserve; what depth buys is headroom before the \$1 cap binds, and once it binds the gain stops responding to payment size at all. Pushing both together, a deeply impaired reserve taking a payment large enough to restore par returns a double digit percentage on the bracketed deposit rather than the fraction of a percent above. The deposit premium adds to the margin directly and, unlike the other two, is chosen by the attacker rather than by the treasury: taken at the 50 bps the band nominally admits it roughly doubles the profit on the sequence above, and it correspondingly lowers the smallest payment worth bracketing, so the set of transactions worth watching is wider than the large treasury payments described here.

That premium figure is measured at exactly \$1.0050, so it does not rest on the separate finding `_The de-peg band admits one basis point more than the redemption fee, making a deposit and redeem round trip profitable_`. That finding shows the enforced band reaches almost a basis point further than the configured one, which widens this margin rather than creating it. Separately, the `reportUnchecked(...)` function of the `EnergyAssetsReporter` contract can also move `C` upward without a band, though the code pairs that path with an operational instruction to pause the desk.

The value extracted comes out of the reserve, so every uWatt holder who did not participate is diluted. The attack is repeatable up to `dailyRedeemLimitBps`, which defaults to 100 bps of supply per day, and each repetition consumes redemption capacity that legitimate holders would otherwise have. The net effect is that the recapitalisation which rescue mode exists to attract is siphoned off by whoever is watching the mempool, in precisely the state where the protocol is most fragile. Two conditions must hold at once: the reserve below par, and `cFloorWad` lowered under `1e18`. Neither is the default, since `cFloorWad` is initialised to `1e18` and blocks minting below par outright. Because minting below par is exactly `C`-neutral, the mint floor check reduces to `cFloorWad <= C`, so any floor at or beneath the prevailing `C` is sufficient and a cautious rescue posture opens the same window as a deep one. Both are an intended and documented operating mode, and once active the attack is permissionless.

**Recommendation(s):** Consider restoring a bound between the price used to mint and the price used to redeem while the cap is not binding. The existing `maxPriceJumpBps` and `lastPriceWAD` state is still maintained and could be consulted again on the redemption path while `C < 1`, where the argument for retiring it does not hold, and unlike the alternatives below it covers every contributor at once regardless of source.

Alternatively, consider splitting a redemption into a request and a later settlement, with the payout taken as the lesser of the value owed at request time and the value owed at settlement time. That removes atomicity, so the two legs can no longer be placed either side of a single victim transaction, and the asymmetric payout leaves the redeemer carrying any adverse move over the waiting period without

capturing a favourable one. On its own this raises the cost of the attack rather than closing it, since the rise in  $C$  persists and the lesser of the two valuations is still the post inflow one, so the attacker keeps the edge while merely holding it at risk. For the mechanism to close, the same treatment needs to reach the mint side, so that a deposit is priced at whichever of the request time and settlement time ratios is least favourable to the depositor.

Consider also whether an energy payment needs to be recognised in the reserve in full at the instant it lands. The `SunoStakingVault` contract already vests rewards linearly, deducting the unvested portion inside its `totalAssets(...)` function, for structurally the same reason: value recognised instantly is value a participant of one block can capture. Recognising an inflow over a period would remove the step in  $C$  rather than pricing around it. The trade off is that reserve accounting derives entirely from real custody balances today, deliberately and with no manual accumulators, so a vesting schedule reintroduces state that has to be reconciled against those balances, and it addresses the payment specifically rather than the bond and cash terms, which continue to move  $C$  in steps of their own.

The underlying asymmetry is worth revisiting independently of the fix chosen: reporter updates are banded, whereas cash entering custody, the stablecoin mark applied to cash already held, and the bond NAV all move  $C$  upward with no equivalent limit. Consider whether the reasoning that retired the desk band should be re-derived for the ascending direction and documented alongside the descending case it currently covers.

**Status:** Fixed

**Update from the client:** Confirmed, including the two-address bypass of the per-address guard and the worked example — we reproduced it in our own suite: with the reserve impaired to  $C = 0.9333$  and rescue minting open, an atomic mint  $\rightarrow$  hand-off  $\rightarrow$  redeem around a \$50k custody inflow returned **+327.84 USDC (+6.56%) risk-free**. You correctly identified that retiring the desk jump-band was justified only for the direction in which  $C$  `_descends_` (reporter moves, banded); it did not cover the `_ascent_`, which never passes through the reporter —  $C = (\text{energy} + \text{bond} + \text{cash})/\text{supply}$  rises with any cash landing in custody (an energy payment, the un-banded stablecoin re-mark, a bond NAV step), so the fix bounds the payout price itself rather than any one contributor.

**Fix (oracle + issuer):** the payout-price guard now lives in the `uWattPriceOracle`, which bounds the sub-par **payout** price to a per-block checkpoint plus `maxPriceJumpBps` (read from the issuer, the single source of the fee > band invariant). `uWattPriceWAD()` stays raw, so the monitor, the cross-chain composer and the L2 mark are untouched. The mint side is never ceiling-clamped — clamping it would credit more `uWatt` per dollar than the reserve backs — but a falling  $C$  is pushed into the checkpoint on mint too (down-only), so a stale-high checkpoint can't leave the ceiling above a rebound. `acceptPriceJump` moves to the oracle as the escape hatch after a legitimate revaluation. Fixed in [bcd83bf](#).

**Fix (reserve — the second payout path):** the root cause was several paths pricing without a common guard, so the reserve's `pWattSwap` — which also pays value out for `uWatt` burned — now reads the **same** guarded payout price from the oracle, closing the sub-par ascent on that path too. Over par this also stops it pricing at the raw ratio (it burns at \$1, keeping the surplus in the reserve rather than value-neutral). Note this makes `pWattSwap` **surplus-accretive** rather than the previously  $C$ -neutral behaviour, so the reserve is worth a re-review against that changed invariant. In the same commit, and independent of this finding, `swapProject` (the mint direction) gains a governed per-project **retention lever**  $P \in [\$1, C]$ ; its default (unset) reproduces mint-at- $C$  exactly, so it is behaviour-neutral until governance turns it — we're flagging it here explicitly so it gets audit eyes even though no finding requested it. Fixed in [8f9cce7](#).

**Re-review against the improved write-up (depth-limited / over-par regime):** your revised write-up separates the `_payment-limited_` regime — a small inflow that leaves  $C$  sub-par — from the `_depth-limited_` one,  $(1 - \text{fee})/C_{\text{before}} - 1$  once the cap binds, independent of payment size. The guard above closed the first (the  $-10.08$  loss below is that case) but not the second: `_guardedPayout` short-circuited to \$1 the instant  $C$  reached par, abandoning the sub-par ceiling exactly when an inflow — or the un-banded cash re-mark — carried  $C$  back `_over_` par, so a mint at deeply sub-par  $C$  redeemed at the full \$1 on `uWatt` bought cheap. We reproduced it: a genuine \$100k custody inflow crossing par returned **+6.6% risk-free**. The fix applies the checkpoint ceiling **across the par boundary** — `payout = min(\$1, C)` capped by `checkpoint*(1+band)` in `_both_` regimes — so a redeem right after a sub-par mint is throttled to  $C_{\text{before}}$  whether or not  $C$  crossed par. In steady over-par operation the checkpoint sits at \$1, so the ceiling is  $\geq \$1$  and honest redeemers are unaffected; the throttle only bites while the checkpoint catches up through par after a genuine recovery, where `acceptPriceJump` snaps it. Fixed in [48376c1](#).

The original reproduction, run against the fixed code, now returns a  **$-10.08$  USDC loss** — the bracket pays the fee. Residual, stated plainly: a bracket `_held_` across  $k$  blocks can still capture  $k \cdot \text{band} - \text{fee}$ , which is a position carrying market exposure over the wait, not a risk-free atomic arbitrage; governance tunes the rate and `acceptPriceJump` snaps the checkpoint after a genuine recovery. Verified in isolation (10 tests on the oracle guard) plus adversarial edges: the bounded residual, the over-par  $\rightarrow$  sub-par transition not overpaying, the throttle-then-`acceptPriceJump` path, and no-mint & honest-redeem variants; and on the reserve side, `pWattSwap` burning at \$1 over par (surplus retained) and the retention lever at 0 / 5000 / default. The rollout is deployed and exercised on Sepolia (oracle v4 + issuer **and** reserve upgrades; live mint  $\rightarrow$  redeem and `pWattSwap` round-trips confirm the guarded paths). `_branch fix/audit-round-1_`

**Self-review — the reserve mint path did not seed the checkpoint.** The guard's low-water-mark checkpoint is what stops a cheap sub-par mint from being redeemed into a recovery — `depositAndMint` pushes it DOWN to the  $C$  it mints at (`syncMintPrice`), so a later redeem is throttled to  $C_{\text{mint}} \cdot (1 + \text{band})$ . Reviewing the guard end-to-end we found the reserve's `swapProject` — the `_other_` mint path — priced via a view (`rawPriceWAD`) and never advanced the checkpoint, so `uWatt` minted there at a low  $C$  was not recorded: with a stale-high checkpoint, a holder swapped in cheap could later redeem capturing a recovery the guard throttles for depositors. It is **not** a risk-free atomic bracket (`swapProject` is `SWAP_OPERATOR_ROLE`-gated, so neither leg is attacker-timed) — the same "held across blocks carries exposure" residual — but it broke the guard's uniformity across mint paths. Fix: `swapProject` now calls `syncMintPrice` for its down-only side effect, seeding the checkpoint at the sub-par  $C$  exactly like the deposit desk (inert over par, so the retention lever is untouched; it cannot live in the shared `_mintPrice`, which `previewSwap` calls as a view). Red  $\rightarrow$  green: a stale-high checkpoint,  $C$  dropped sub-par, a swap now pushes the checkpoint down to  $C$  (before, it stayed stale-high). Fixed in [5ef8b92](#) (`_branch fix/audit-round-1_`)

**Update from the Nethermind security team:** One concern remains on the residual. The payout ceiling advances once per block on any redemption regardless of size, and a dust redemption clears every desk check while paying nothing out and consuming a negligible slice of the daily limit, so the ceiling can be walked up to the recovered price for the cost of gas rather than waiting on unrelated desk activity. We reproduced the delayed bracket against the fixed code and it exits for a double-digit percentage gain on the bracketed deposit after a few minutes of dust redemptions. It is not atomic, but the gain is already realised in the collateral ratio once the payment lands, so the exposure is an adverse move inside a wait the attacker sets the length of. Bounding this at the price layer would need the ceiling to advance on something that cannot be supplied for free, such as redeemed volume or a minimum redemption size, though our preference

is still to remove the discrete step so there is no jump to time a bracket around at any block count. Accepting the current behaviour is reasonable if inflows of that size relative to the reserve are not expected while rescue minting is open, and we would only suggest stating that assumption, since the per-block band governs how long an attacker waits rather than how much is extracted. We also noticed that the deposit and redemption paths no longer reject a zero price, which the pre-fix helper failed closed on. The oracle guard rejects only a stale reporter and a zero supply. The state is remote, as it needs the reserve to value to nothing against a live supply, but in it a redemption with no slippage bound burns uWatt for no payout and the zero written into the checkpoint falls back to the bootstrap branch that drops the ceiling, so an explicit check on both paths may be worth restoring.

**Update from the client (re-review):** Both notes taken: the first closed in code, the second accepted as a residual.

Zero price. Correct: `_rawPrice()` rejected only a stale reporter and a zero supply, so a zero-valued reserve against live supply let `redeemUWatt` burn for a zero payout and `syncPayoutPrice()` write 0 into the checkpoint, dropping the ceiling for everyone (mint only panicked on the division). Restored as if `(rawC == 0)` revert `ZeroPrice()` on the desk paths (`_guardedPayout`, `peekMintPrice`, `syncMintPrice`, `acceptPriceJump`), reverting before any checkpoint write; `uWattPriceWAD()`/`rawPriceWAD()` stay raw so the monitor and L2 composer observe the zero and alarm. Fixed in [82d593b](#) (branch `fix/audit-round-1`).

Dust-walk residual. Confirmed: `redeemUWatt` has no minimum size, so dust advances the checkpoint by the band each block and walks it to the recovered price for gas. We accept it with the assumption stated: whenever `C` is sub-par (the checkpoint below the raw ratio, from rescue minting or real impairment), no material `C`-raising inflow lands on a live desk. Per the risk-posture runbook, `distributeEnergyPayment`, bond unwinds and upward re-marks run under `pause()` and are paired with `acceptPriceJump()` before unpausing, and the same rule covers re-enabling a funded deposit asset. Removing the discrete step at the root (inflow vesting, or request/settle with lesser-of pricing) is an R2 design point; the cheaper variants (minimum size, volume-proportional advance, governance-only ratchet) raise the floor without closing it. This part is an accepted residual, not a code change.

## 6.2 [Medium] Hardcoded USDC value in the YieldController contract dilutes protocol collateralization

**File(s):** /contracts/core/YieldController.sol

**Description:** The `distributeEnergyPayment(...)` function from the `YieldController` contract processes incoming energy payments and splits them between the protocol reserve and the staking vault based on the protocol's collateralization ratio.

To compute the USD value of the incoming USDC, the function scales the 6 decimals amount by  $1e12$ . This hardcodes the value of one USDC to \$1.00. However, the protocol's core NAV calculation, relying on the `reserveValue(...)` function from the `uWattIssuer` contract, correctly prices the newly deposited USDC using its live Chainlink oracle price.

```

1  function distributeEnergyPayment(uint256 usdcIn) external {
2      // ...
3      // @audit-issue The function assumes 1 USDC is $1.00 by scaling it
4      // without querying the oracle.
5      uint256 Y = usdcIn * T018;
6
7      // ...
8      // @audit The split calculation relies on the hardcoded Y value.
9      (uint256 reserveAmount, uint256 vaultAmount) = _computeSplit(Y);
10     // ...
11 }
12

```

The `previewSplit(...)` function also uses the same parity mismatch:

```

1  function previewSplit(uint256 usdcIn) external view returns (uint256) {
2      // ...
3      // @audit-issue The calculated Y value diverges from the actual value
4      uint256 Y = usdcIn * T018;
5      // ...
6  }
7

```

When USDC trades below \$1.00 (e.g., \$0.99), the actual USD value added to the protocol's reserve is less than the computed  $Y$ . Because the yield split calculation relies on the overstated  $Y$  to determine the distributed amount, it mints an inflated amount of `uWatts` to the staking vault. These excess `uWatts` are unbacked, which dilutes the protocol's collateralization ratio.

Conversely, if USDC trades above \$1.00, the  $Y$  value is understated relative to the true value added to the reserve, causing the controller to mint fewer `uWatts` than it should.

**Recommendation(s):** Consider querying the live Chainlink oracle price for USDC to calculate the accurate USD value of the incoming payments, ensuring it remains consistent with the protocol's core NAV calculations.

**Status:** Fixed

**Update from the client:** Confirmed, this is the behavior we discussed: `distributeEnergyPayment/previewSplit` valued the payment at par while `reserveValue()` prices the same custody USDC at the live oracle, so below par the split ran on an overstated  $Y$  and minted unbacked `uWatt` to the vault (the unsafe direction — on the redemption side the par convention `_under_` pays, which is conservative). The fix values the payment at `min(oracle price, $1)` via a `_usdValue()` helper shared by both functions. The `min` is a deliberate divergence from "use the live price": an above-par stable must not inflate distributions either — retaining more is always the safe direction (pinned by test). We added no local staleness check because none is needed: the split's own `reserveValue()` read prices the just-landed custody USDC through `_stablePriceFresh`, which already fails the entire payment closed on a stale or zero feed — a regression test pins that inheritance. Red→green repro: a \$0.98 payment now splits exactly on  $0.98 \cdot Y$  (previously over-minted by  $1/0.98$ ); at \$1.02,  $Y$  stays capped at par. Fixed in [d4f3cc9](#) (`_branch fix/audit-round-1_`)

### 6.3 [Medium] Inconsistent valuation of custody USDC moves the collateral ratio spuriously, and the move can be sandwiched

**File(s):** `contracts/UWattIssuer.sol`, `contracts/UWattIssuer.sol`, `contracts/adapters/SkySUSDSAdapter.sol`

**Description:** The UWattIssuer contract measures the reserve as energy assets plus the bond tranche plus the cash in the custody Safe. The collateral ratio C is that total divided by the uWatt supply, and it drives the desk: it gates minting through cFloorWad, it prices both legs once the reserve falls below par, and the YieldController contract reads it to size staker distributions.

Cash is valued at whatever the stablecoin feed reports:

```

1 function cashValue() public view returns (uint256 usd) {
2     // ...
3     // @audit-issue Custody USDC is valued at the ORACLE price, not at one dollar.
4     uint256 p = _stablePriceFresh(asset);
5     usd += (IERC20(asset).balanceOf(custody) * p) / (10 ** cfg.decimals);
6 }

```

Bonds are valued on a different basis. The SkySUSDSAdapter contract prices sUSDS at the rate the position would really exit at, and its documentation calls that the conservative choice: valuing at the exit rate rather than assuming USDS is worth a dollar, which it says keeps the NAV "consistent with what the reserve would really receive":

```

1 function navWAD() public view returns (uint256) {
2     uint256 usdsPerShare = sUsds.convertToAssets(WAD);
3     // @audit-issue The exit is denominated in USDC, so the bond is measured in USDC
4     // units taken at one dollar each.
5     return (usdsPerShare * WAD) / (WAD + psm.tout());
6 }

```

That reasoning stops one conversion short. The exit it values against pays out USDC, and the Sky PSM swaps USDS for USDC one for one whatever USDC is trading at. So the bond tranche is USDC units counted at a dollar while the cash tranche is the same units counted at the oracle price. The purchase path takes the par view too, sizing its payment by dividing dollars by T018, the constant  $1e12$ :

```

1 uint256 usdWAD = (pWattAmount * price) / WAD;
2 // @audit-issue The payout converts dollars into USDC at par, while the USDC that
3 // leaves custody was marked at the oracle price.
4 usdcOut = usdWAD / T018;

```

Because the bases disagree, any transaction moving value between them changes the reported reserve while the protocol's holdings are economically unchanged. The `deployCash(...)`, `redeemBond(...)` and `acquireProjectTokens(...)` functions all do this. None of them transacts at a market determined price, and none charges a fee at current PSM settings, yet each steps the reserve by the deviation on the amount converted. The sign depends on direction: `deployCash(...)` and `acquireProjectTokens(...)` send out cash marked at  $p$  and receive value counted at par, stepping the reserve by converted  $\times (1 - p)$ , while `redeemBond(...)` runs the other way and steps it by converted  $\times (p - 1)$ . So with USDC above the peg an unwind converts par-valued bond into USDC marked above par and the reserve **rises**, even though nothing entered it.

That matters because governance can run the protocol below par by lowering the `cFloorWad` parameter, the "rescue floor", so deposits can recapitalise an impaired reserve. In that mode the  $\min(\$1, C)$  cap stops binding and  $C$  is the live price on both legs, so a spurious rise in  $C$  is a spurious rise in what the desk pays, and it can be bracketed by a deposit and a redemption.

The redemption fee looks like it should absorb this, and the `setDepositAsset(...)` function requires `maxDeviationBps <= redemptionFeeBps` with exactly that intent. That guard does bound the plain round trip, but it does so by **exact cancellation**, and that is what leaves the exposure open here. A round trip returns  $p \times (1 - \text{fee})$ , so at a 50 bps premium against a 50 bps fee it returns  $0.99997500$ , a loss of only  $\text{fee}^2$ . Every figure below is taken at exactly  $\$1.0050$ , the premium the band nominally admits, so none of it rests on the separate finding `_The de-peg band admits one basis point more than the redemption fee, making a deposit and redeem round trip profitable_`. That finding shows the enforced band reaches almost a basis point further than the configured one, which lowers this bar further still; correcting it leaves the numbers here unchanged. The fee has already been spent neutralising the premium. Writing  $dc / C$  for the relative change in the collateral ratio a single transaction produces, the bar falls roughly one for one as the deposit premium rises, from 50 bps at the peg to 0.25 bps at a 50 bps premium.

That does not by itself make the attack cheap, because the spurious  $C$  move scales with the same premium. With  $k$  the fraction of the reserve converted, the move is  $k \times \text{premium}$  while the bar is about  $\text{fee} - \text{premium}$ , so the round trip pays only once  $\text{premium} > \text{fee} / (1 + k)$ . Since  $k$  cannot exceed one, that floors the whole class at 25 bps of premium.

Only the `redeemBond(...)` function lines the two effects up: it raises  $C$  when USDC is **above** the peg, which is also when the deposit leg is cheapest. The other two raise  $C$  only below the peg, where the deposit credit is worse and the bar rises past 100 bps. Bracketing a `deployCash(...)` call of 80% of custody cash with USDC at  $\$0.9950$  lost 87.46 USDC on the same 10,000 deposit, and the equivalent bracket around an `acquireProjectTokens(...)` call also lost, so the exposure demonstrated here is on the unwind path. A concrete sequence, with part of the reserve in sUSDS, `cFloorWad` lowered so the desk is below par, and USDC reading  $\$1.0050$ :

1. The attacker observes a pending call to the `redeemBond(...)` function unwinding 400,000 of the sUSDS position;
2. Front running it, attacker address A deposits 10,000 USDC, credited at the oracle price as 10,050 USD, minting  $10,050 / C$  uWatt;

3. Address A hands the uWatt to attacker address B, defeating the per address round trip guard, a bypass the contract documents as being "sidestepped with a second account";
4. The unwind lands. 400,000 of bond carried at par becomes 400,000 USDC that the cash tranche marks at \$1.0050, so the reserve gains 2,000 USD that never existed. Against a post-deposit reserve of 3,262,550 USD that is a relative rise in C of 6.13 bps, versus a 0.25 bps bar at this deposit price;
5. Address B redeems at the new C and receives 10,005.88 USDC;

The attacker spent 10,000 and recovered 10,005.88, a profit of 5.88 bps with no market exposure. Running the identical deposit and redemption **without** the unwind returns 9,999.75 USDC, a loss of 0.25, which confirms the gain comes from the C move rather than from the deposit premium alone.

The precondition on the USDC price is the binding one. Sweeping the deposit price with everything else fixed, and 400,000 of the 3,262,550 reserve unwound, the round trip only turns positive near the top:

|   |                |    |                |
|---|----------------|----|----------------|
| 1 | premium 40 bps | -> | -5.300097 USDC |
| 2 | premium 45 bps | -> | +0.289705 USDC |
| 3 | premium 50 bps | -> | +5.880020 USDC |

The crossover therefore sits between 40 and 45 bps, which is where the first order estimate  $fee / (1 + k)$  puts it for this k of 0.123, namely 44.5 bps. So the conditions are: the reserve below par with cFloorWad lowered, part of the reserve in bonds, a treasury unwind pending in the mempool, and USDC trading at least a quarter of a percent **above** the peg. The last is the least likely: USDC is redeemable one for one with its issuer, so a sustained premium of that size tends to be arbitrated away by minting, and historical de-pegs have overwhelmingly been to the downside. It is repeatable up to dailyRedeemLimitBps, and each repetition consumes redemption capacity that real holders would otherwise have. What distinguishes this from ordinary front running is that the value captured is not real, since the C increase is an artifact of the reserve valuing the same USDC two different ways, so the reserve is drained to pay out a gain the protocol never made.

**Recommendation(s):** Consider settling on a single basis for the reserve and applying it everywhere. Since the desk's obligation is denominated in dollars but discharged in USDC at par, and every internal conversion rail also transacts at par, valuing custody USDC at par would align the reported reserve with what the protocol can realise through its own rails and leave all three treasury operations value neutral, which addresses the step being sandwiched rather than only making it harder to reach. If marking cash to the oracle is preferred so a genuine de-peg stays visible, then consider carrying that same price through the paths that spend or convert it, so that a conversion no longer moves the total on its own. It may also be worth revisiting the reasoning on the navWAD(...) function, which correctly aims to value the bond at what the reserve would really receive, and noting that the amount received is USDC whose own valuation is then decided elsewhere. Independently of the fix chosen, consider that the  $maxDeviationBps \leq redemptionFeeBps$  invariant leaves only  $fee^2$  of margin once a deposit is taken at the edge of the band, so any future mechanism that can move C below par will face that same 0.25 bps bar rather than the 50 bps the fee appears to provide.

**Status:** Fixed

**Update from the client:** Confirmed — cashValue() marked custody USDC at the \_raw\_ oracle price while every internal rail ( deployCash / redeemBond / acquireProjectTokens) and the redemption payout convert at \_par\_, so an above-peg print revalued the whole cash tranche and moved C while the protocol's holdings were unchanged. As your redeemBond example shows, once the sub-par cap stops binding that spurious rise is sandwichable — a bond unwind marks par-valued bond into above-par cash and C jumps; we reproduced the direction.

**Fix (cash mark):** custody cash is now marked at  $\min(\text{oracle}, \$1)$ . An above-peg print can no longer inflate C — the monetizable direction — while below par the raw price still flows through, so a genuine de-peg stays visible and \_understates\_ the reserve, the conservative direction and the same freshness-only stance cashValue already documents. We took  $\min(\$1)$  over pure par (which would hide a real de-peg) and over carrying the oracle through every rail (a larger change): the residual below-par mismatch between cash-at-oracle and the par rails sits in the non-monetizable direction your own sweep showed losses (the deployCash / acquire brackets both lost).

**Fix (deposit credit, symmetric):** the deposit credit in depositAndMint and previewMint is now also  $\min(\text{oracle}, \$1)$ . This removes the last piece of the sub-par sandwich in the **[High]** — a deposit taken at the top of the de-peg band no longer mints a premium that stacks with the payout jump band in rescue mode (the +0.30% single-block residual that remained after the across-par ceiling fix). It also strengthens the **[Low]** de-peg band round trip: at the band edge it now loses the \_full fee\_ rather than only  $fee^2$ , and that test is updated accordingly — a \$1.005 deposit mints exactly par.

Red→green: a \$1.005 deposit mints exactly par (no premium); the redeemBond crossing-par bracket and the combined premium-plus-ascendant bracket both return a loss. Fixed in [31f6a05](#). \_ (branch fix/audit-round-1)\_

**Update from the Nethermind Security Team:** The cap on cashValue() closes the monetizable direction, but the valuation bases still disagree below par. bondValue() reads no stablecoin oracle, and deployCash(...), redeemBond(...), acquireProjectTokens(...) and the redemption payout still convert at par, so with USDC under the peg each of those conversions continues to step the reported reserve, and therefore C, by the deviation on the amount converted while the protocol's holdings are unchanged. We did not find a profitable round trip against the remaining step, as the desk pays out in the same discounted USDC at par and the deposit credit takes the same raw price. The inconsistency itself remains, and since C gates minting, prices both legs below par and sizes the YieldController split, we would recommend keeping it in view rather than treating the valuation as settled.

**Update from the client (re-review):** Agreed, keeping it in view. The below-par mismatch isn't monetizable (the desk pays out in the same discounted USDC at par and the deposit credit takes the same raw price), but since C gates minting and sizes the yield split, the root-cause fix is an R2 design point: value the canonical USDC at par by definition, unifying all three bases (cash, the par rails, bondValue) rather than only capping the monetizable direction. We'd take that with re-review.

## 6.4 [Medium] The risk posture sizing rule understates the collateral ratio needed to absorb the design shock

**File(s):** [contracts/UWattIssuer.sol](#), [contracts/UWattIssuer.sol](#), [contracts/UWattIssuer.sol](#)

**Description:** uWatt is backed by a reserve of cash, bonds and tokenized solar projects. Solar is the volatile leg, so the protocol holds more reserve than it owes and that surplus is what absorbs a fall in the solar valuation. Three governance dials describe the arrangement: solarCapBps bounds how much solar the reserve may hold, designShockBps is the fall in solar value the protocol declares it can absorb, and collateralTargetWad (called C\* in the code) is the reserve held per dollar owed. The setRiskPosture(...) function is the only place their mutual consistency is enforced, and it documents that intent as "Enforces the sizing rule so no unsafe combination is configurable".

The rule is implemented as an additive expression:

```
1 // @audit-issue The required ratio is computed as 1 + designShock * solarCap, which
2 // treats solarCapBps as a fraction of the uWatt liability.
3 uint256 requiredCStar = WAD + (WAD * designShockBps_ * solarCapBps_) / (BPS * BPS);
4 if (collateralTargetWad_ < requiredCStar) {
5     revert InconsistentRiskPosture(requiredCStar, collateralTargetWad_);
6 }
```

That form is only correct if solarCapBps bounded solar as a fraction of the uWatt liability. It is enforced against the reserve instead, inside the acquireProjectTokens(...) function:

```
1 uint256 energyAfter = energy + usdWAD;
2 // @audit-issue The ceiling is a fraction of the reserve, which exceeds the liability
3 // by exactly the collateral ratio.
4 if (energyAfter * BPS > uint256(solarCapBps) * reserveValue()) {
5     revert SolarCapExceeded(energyAfter, reserveValue());
6 }
```

Because the reserve exceeds the liability by exactly the collateral ratio, permitted solar exposure is solarCap \* C\* \* S rather than solarCap \* S, so the loss the surplus must absorb is larger than the rule assumes by that same factor. Solving the condition that the protocol stays fully backed after the declared shock, with S the uWatt supply, R the reserve and E the solar valuation:

```
1 R - designShock * E >= S,    with    E <= solarCap * R
2
3 R * (1 - designShock * solarCap) >= S
4
5 C* >= 1 / (1 - designShock * solarCap) = 1 / 0.68 = 1.4706
```

The genesis posture set by the initialize(...) function sits inside the gap between 1.32 and 1.4706, and the accompanying comment shows the additive rule being applied by hand:

```
1 // @audit-issue The sizing rule is evaluated against the wrong base.
2 // Genesis risk posture (concentrated portfolio, plan table): C*=1.35,
3 // solar<=80%, liquid>=20%, design shock 40%, mint floor at par.
4 // Sizing rule: 1 + 0.40*0.80 = 1.32 <= 1.35
5 collateralTargetWad = 1.35e18;
6 solarCapBps = 8_000;
7 liquidFloorBps = 2_000;
8 designShockBps = 4_000;
```

Walking the declared shock through those defaults:

```
1 S = 100 uWatt          liability = $100
2 C* = 1.35              reserve   = $135
3 solar <= 80% of R      solar    = $108
4 40% shock on solar     loss     = $43.20
5                       reserve   = $91.80    ->    C = 0.918
```

The reserve ends at \$91.80 against \$100 owed, so the exact scenario the posture was configured to absorb leaves uWatt under backed by roughly 8%, and the guard accepts the configuration because 1.35 clears the understated bound of 1.32. The gap widens as the dials get riskier: at a 50% shock over an 80% cap the guard asks for 1.40 where 1.6667 is required.

The impact is that the solvency property the protocol documents does not hold for the posture it ships with, nor for any posture governance can configure while relying on the guard. A solar write down within the declared tolerance pushes the collateral ratio under par instead of being absorbed, and since holders redeem at min(\$1, C) they exit below a dollar. Note also that the initialize(...) function assigns these dials directly rather than routing them through setRiskPosture(...), so the shipped defaults are never checked at all, though the check would have passed them regardless.

**Recommendation(s):** Consider deriving the minimum collateral ratio from the same base the solar ceiling is actually enforced against, using the relation obtained by solving the post shock backing condition rather than the additive approximation, and consider re-deriving

the genesis posture from the corrected rule and subjecting the initial dials to the same validation. Note that under the existing  $2 * WAD$  upper bound on `collateralTargetWad` the corrected rule is satisfiable only while `designShockBps * solarCapBps` stays below 50%, so the acceptable ranges of those two dials are worth reviewing alongside it.

**Status:** Fixed

**Update from the client:** Confirmed, including the worked example — the shipped genesis posture ( $C^*=1.35$ , solar 80%, shock 40%) leaves  $C$  at **0.918** after the exact shock it declares it can absorb, so holders redeem at  $\min(\$1, C)$  8% below par. The additive rule  $1 + \text{designShock} * \text{solarCap}$  is the answer for a `_liability_`-based cap; the ceiling is enforced against the `_reserve_` ( $E \leq \text{solarCap} * R$  in `acquireProjectTokens`), so at the cap the loss is  $\text{shock} * \text{solarCap} * R = \text{shock} * \text{solarCap} * C * S$  — larger than the liability by exactly  $C$ .

**Fix:** the sizing rule is now derived from the base the ceiling is actually enforced against. Solving  $R - \text{shock} * E \geq S$  with  $E \leq \text{solarCap} * R$  gives  $C^* \geq 1 / (1 - \text{designShock} * \text{solarCap})$  — the reserve-based rule, in a shared `_requiredCStar` helper used by both `setRiskPosture` and `initialize`. The  $\text{shock} * \text{cap} == WAD$  edge is guarded against division by zero; combos with  $\text{shock} * \text{cap} \geq 50\%$  need  $C^* > 2 * WAD$  and are rejected by the existing  $C^*$  upper bound.

**Genesis re-derived and validated:**  $C^*$  raised **1.35 -> 1.5** ( $\geq$  the required 1.4706, 2% headroom above par post-shock), and `initialize` now subjects the genesis dials to the same rule — they were never checked before. Finance may ratify a different point (the exact minimum 1.4706 lands at par; a lower cap trades solar for headroom). Red→green: the former genesis posture is now rejected, the corrected boundary  $1 / (1 - \text{shock} * \text{cap})$  binds to the wei, and the stage-transition postures are re-derived to the reserve-based rule. Fixed in [d48e540](#) (`_branch fix/audit-round-1_`)

## 6.5 [Low] Disabled deposit assets break the protocol's cash valuation and operations

**File(s):** [contracts/UWattIssuer.sol](#)

**Description:** The `cashValue()` function from the `UWattIssuer` contract is used to calculate the aggregate cash value of the reserve. It does this by iterating over every asset in the `depositAssetList` and fetching a fresh price from the stablecoin oracle.

For each asset, it calls the `_stablePriceFresh(...)` function, which enforces a strict staleness check using `maxStableStaleness`. However, the `cashValue()` function does not check if the asset is currently enabled (`cfg.enabled`).

```

1  function cashValue() public view returns (uint256 usd) {
2      uint256 len = depositAssetList.length;
3      for (uint256 i; i < len; ++i) {
4          address asset = depositAssetList[i];
5          DepositAssetConfig memory cfg = depositAssets[asset];
6
7          // @audit-issue The function does not check if the asset is enabled.
8          uint256 p = _stablePriceFresh(asset);
9          usd += (IERC20(asset).balanceOf(custody) * p) / (10 ** cfg.decimals);
10     }
11 }
12

```

If governance decides to remove an asset by setting its `enabled` flag to `false`, the asset remains in the `depositAssetList` because there is no function to remove items from this array. As a result, the `cashValue()` function will continue to check the price feed for the deprecated asset.

Once the deprecated asset's oracle feed stops updating or is abandoned, the feed will cross the `maxStableStaleness` threshold, causing `_stablePriceFresh(...)` to consistently revert. Because `cashValue()` reverts, `reserveValue()` will also revert, which fundamentally breaks the entire `UWattIssuer` contract. All critical functions, including `depositAndMint(...)`, `redeemUWatt(...)`, `deployCash(...)`, and `YieldController` distributions, will suffer a DoS since they all rely on `reserveValue()` to function.

**Recommendation(s):** Consider updating the loop inside the `cashValue()` function to explicitly skip disabled assets or skip the staleness check for disabled assets if they hold a zero balance.

**Status:** Fixed

**Update from the client:** We confirmed the DoS: entries never leave `depositAssetList`, so a dead feed on a deprecated asset reverted `cashValue()` → `reserveValue()` → the whole desk. Our first fix skipped the freshness read on zero custody balance — in our own adversarial re-review we found it re-opened the DoS as repeatable third-party griefing (anyone could donate 1 wei of the dead-feed asset to the custody Safe, forcing the stale read again), so we amended it. The skip now keys on the governance flag plus the zero-balance fast path: `if (!cfg.enabled || bal == 0) continue`. `enabled` only moves under `COLLATERAL_ADMIN`, so no third party can force a dead feed back into the loop (a griefing-repro test donates 1 wei of a dead-feed deprecated asset and asserts the desk stays alive). A disabled asset is not valued while it drains (understating is the safe direction; runbook is drain-then-disable, re-enabling restores the valuation); an enabled asset with a balance and a dead feed still fails closed, and disabling it is now the emergency valve. This matches the first option of your recommendation. Fixed in [0978583](#), amended in [cf3da0b](#) (\_branch fix/audit-round-1\_)

**Update from the Nethermind Security Team:** The contract-level fix is correct, but it moves part of the safety property into the operator runbook, which the response describes ambiguously ("a disabled asset is not valued while it drains" against "runbook is drain-then-disable"). Nothing on-chain enforces the drain, so toggling `enabled` on a funded asset is a single-transaction step in `cashValue()` and therefore in C. Downward, a redeem during the depressed window pins the oracle payout checkpoint low and honest redeemers stay throttled below par after the asset is re-enabled. Upward, with the rescue window open (`cfloorWad < 1e18`), the restored step is the ascending move in C that the [High] covers, capturable by minting into the depressed window, and immediately so if `acceptPriceJump()` is used to restore the price. We recommend that the ordering of these operations be settled deliberately rather than left implicit, and that the same care be applied to any other privileged action that moves the valuation in one transaction. We would also note that an asset left enabled with a zero custody balance and a stale, retired or unconfigured feed is protected only by the `bal == 0` fast path. Anyone can donate 1 wei to custody to force the read back into the loop, reverting `cashValue()` and with it `reserveValue()`, `depositAndMint(...)`, `redeemUWatt(...)`, `previewRedeem(...)`, `swapProject(...)`, `pWattSwap(...)` and `distributeEnergyPayment(...)` until `COLLATERAL_ADMIN` disables it. "Enabled but empty" is therefore not a safe resting state, since it is the governance flag rather than the balance that keeps such an asset out of the valuation.

**Update from the client (re-review):** Both notes taken. No contract change is needed; both live at the runbook and monitor layers.

On ordering. Agreed that our response described drain-then-disable ambiguously, and that nothing on-chain enforces the drain: toggling `enabled` on a funded asset is a one-transaction step in `cashValue()` and therefore in C, sandwichable in the directions you describe. The risk-posture runbook now settles the order deliberately. Every enabled toggle on an asset with custody balance above zero runs under `pause()` and, if it moves C materially, is paired with `acceptPriceJump()` before unpausing, the same rule we adopted for rescue-window inflows on the [High]. For the canonical USDC the contract now makes it undisableable outright (see the USDC question in the follow-up); for a secondary stable, keeping that toggle deliberate is an operational rule today.

On "enabled but empty". Agreed it is not a safe resting state: the governance flag, not the balance, is what keeps an asset out of the valuation, and a 1-wei donation reopens the DoS until `COLLATERAL_ADMIN` disables it. Two closures. The runbook now states that deprecating an asset means draining and disabling it in one procedure, never leaving it enabled without funds. And the monitor gained a check (`deposit-asset-resting-state`): for each enabled asset with zero custody balance it reads the feed, and a stale or reverting feed raises a WARN naming the asset, before the donation rather than after. WARN rather than CRITICAL on purpose, since nothing is lost yet

and a public address must not be a pause lever. Tests: the state is silent on-chain ( `cashValue()` still computes via the fast path), so the monitor is the only thing that surfaces it; disabling clears it; the baseline stack is not flagged.

## 6.6 [Low] Dynamic daily redemption limit restricts withdrawals more than intended

**File(s):** `contracts/UWattIssuer.sol`

**Description:** The `redeemUWatt(...)` function from the `UWattIssuer` contract is used by users to redeem their `uWatt` tokens. To protect the protocol from unexpected outflows, the function enforces a daily redemption limit based on the total supply of the `uWatt` token and a basis points configuration ( `dailyRedeemLimitBps`).

The limit is dynamically calculated using the current `totalSupply()` of the token. As users redeem their tokens throughout the day, the total supply decreases, which causes the absolute daily limit to shrink.

```

1  function redeemUWatt(...) external {
2      // ...
3      // @audit The limit is calculated dynamically based on the current total supply.
4      uint256 limit = (uWatt.totalSupply() * dailyRedeemLimitBps) / BPS;
5      // ...
6      // @audit-issue The accumulator monotonically increases while the limit dynamically decreases.
7      if (redeemedUWattPerDay[day] + amount > limit) {
8          revert DailyRedeemLimitExceeded();
9      }
10
11     redeemedUWattPerDay[day] += amount;
12     // ...
13 }
14

```

While the limit shrinks dynamically, the `redeemedUWattPerDay[day]` accumulator remains absolute and continuously increases with each redemption. If the protocol experiences a high volume of redemptions in a single day, the shrinking limit will prematurely cross below the accumulated redeemed amount. This can lead to an operational DoS of the redemption desk, causing all subsequent redemptions to revert with the `DailyRedeemLimitExceeded` error.

**Recommendation(s):** Consider snapshotting the total supply at the start of each UTC day and calculating the daily limit based on that static snapshot.

**Status:** Mitigated

**Update from the client:** We confirmed the effective cap degraded to  $b/(1+b)$  of day-start supply. Instead of the recommended snapshot, the limit base is now reconstructed as `totalSupply() + redeemedUWattPerDay[day]`. The day-start supply net of the desk's own burns, reusing a storage read the function already performs (no new storage; the issuer is 200 bytes from EIP-170). Intraday mints grow the base (the cap is proportional to backed supply by design, and the same-block round-trip guard prevents mint→redeem gaming); burns outside the desk still shrink it, which is the conservative direction. Red→green test: sequential redemptions now reach the exact configured bps of day-start supply, and the cap still binds there. Fixed in [71170a0](#) (`_branch fix/audit-round-1_`)

**Update from the Nethermind Security Team:** The fix is applied to the `redeemUWatt(...)` function. However, the limit is still derived from the total supply, so the `depositAndMint(...)`, `distributeEnergyPayment(...)` and `swapProject(...)` functions can increase it during the day, while the `pWattSwap(...)` function can decrease it. A decrease is the more relevant case, since the amount already redeemed keeps growing while the limit drops, which can reintroduce the premature revert described above. As all of these functions are restricted to privileged roles, except `depositAndMint(...)` which only increases the limit, we consider this issue mitigated, though the effect of `pWattSwap(...)` is worth reflecting in the operational runbook.

## 6.7 [Low] The de-peg band admits one basis point more than the redemption fee, making a deposit and redeem round trip profitable

**File(s):** [contracts/UWattIssuer.sol](#), [contracts/UWattIssuer.sol](#)

**Description:** The UWattIssuer contract prices the two sides of a round trip differently with respect to the stablecoin itself. A deposit is credited at whatever the oracle reports, so a token trading above \$1 mints proportionally more uWatt:

```
1 uint256 stablePrice = _stablePriceChecked(stable, cfg.maxDeviationBps);
2 // @audit The deposit is credited at the oracle price, so a premium mints more uWatt.
3 uint256 depositUSD = (amountIn * stablePrice) / (10 ** cfg.decimals);
```

The redemption converts back into USDC at exactly one dollar per token, with no oracle read, since T018 is the fixed constant 1e12:

```
1 uint256 grossUSD = (uWattIn * price) / WAD;
2 // @audit The payout converts USD to USDC at par, so the premium credited on
3 // the way in is never charged back on the way out.
4 usdcOut = (grossUSD * (BPS - redemptionFeeBps)) / BPS / T018;
```

The protocol is aware of this and guards it in the `setDepositAsset(...)` function, requiring that the premium a deposit accepts never exceeds the fee a redemption charges:

```
1 // Anti-arbitrage invariant: the deposit accepts a premium up to maxDeviationBps
2 // ...
3 // band <= fee makes the round-trip (1+band)(1-fee) <= 1-fee^2 < 1 - never profitable.
4 // @audit The comparison is against the nominal band, but the band actually
5 // enforced reaches almost a full basis point further, as shown below.
6 if (cfg.maxDeviationBps > redemptionFeeBps) revert InvalidDeskParams();
```

The algebra quoted there is correct. The enforcement is not, because the band comparison truncates the deviation to whole basis points before comparing:

```
1 function _stablePriceChecked(address stable, uint16 maxDeviationBps)
2     internal
3     view
4     returns (uint256)
5 {
6     uint256 p = _stablePriceFresh(stable);
7     uint256 diff = p > WAD ? p - WAD : WAD - p;
8     // @audit-issue Integer division truncates the deviation to whole basis points,
9     // so a band of 50 admits every price strictly below 51 bps of deviation.
10    if ((diff * BPS) / WAD > maxDeviationBps) revert StableDepegged(p);
11    return p;
12 }
```

With `maxDeviationBps` at 50, the expression only exceeds 50 once it reaches 51, which needs a deviation of at least 0.0051. Every price below \$1.0051 is therefore admitted, not every price up to \$1.0050. Meanwhile the round trip pays whenever  $p * (1 - \text{fee}) > 1$ , which for a 50 bps fee means  $p > \$1.005025126$ . Prices in  $[\$1.005025126, \$1.0051)$  are both accepted and profitable. The reference deployment sits on this boundary: the `deploy.ts` script registers USDC with `maxDeviationBps` of 50, and `redemptionFeeBps` is initialised to 50.

Against that configuration, with the reserve over-collateralised so both legs price at the \$1 cap:

1. The USDC feed reports 100509999 on its 8 decimal scale, which is \$1.00509999, the highest reading the band admits;
2. The attacker calls the `depositAndMint(...)` function with 10,000 USDC. It is credited as 10,050.9999 USD and mints 10,050.9999 uWatt;
3. In any later block the same address calls the `redeemUWatt(...)` function for that balance. The desk deducts the 50 bps fee and pays out 10,000.744900 USDC at par;

That is a profit of 0.7449 USDC. Raising the feed one unit to 100510000 makes the deposit revert with `StableDepegged`, and lowering it to 100500000, the deviation the band nominally permits, makes the same round trip lose 0.25 USDC, which is the  $\text{fee}^2$  the invariant intends. The window is created by the truncation alone.

The per address round trip guard is not a mitigation here. The redemption leg never reads the stablecoin oracle, so the premium is banked at mint time and the exit is insensitive to what the stablecoin does afterwards: the two legs need not share a block, and one address suffices.

The impact is a permissionless and risk free withdrawal from custody whenever USDC's oracle reads inside that window, diluting holders who did not participate. The magnitude is small: profit is capped near 0.745 bps of the amount round tripped, throughput is limited by `dailyRedeemLimitBps`, and it requires USDC to read above \$1.005, which is an unusual print. The reason to address it is that a guard written specifically to make this round trip unprofitable does not do so at the setting the project ships with. The same nominal comparison is repeated in the `setDeskParams(...)` function, which re-validates every registered asset against a new fee, so both entry points inherit it.

**Recommendation(s):** Consider performing the de-peg comparison without truncating to whole basis points, for instance by scaling the configured band up to WAD and comparing against the raw deviation, so the band enforced is the band configured. Measured against the current parameters, that alone closes the window observed here: with the comparison made exactly, the most favourable admitted reading leaves the round trip at a loss of  $\text{fee}^2$ , and a sweep across the admitted band produced no profitable reading. Consider also whether the admissible band should follow the exact break-even, since the round trip only stops paying while the premium is strictly below  $\text{fee} / (1 - \text{fee})$ ; requiring the band to be strictly less than the fee would leave a margin rather than resting entirely on the  $\text{fee}^2$  term. Care is warranted in how the redemption leg is changed, because not every way of making it oracle aware is safe: converting the payout at the oracle price outright would widen the bound to  $(1 + \text{band}) * (1 - \text{fee}) / (1 - \text{band})$ , so a deposit at the top of the band and an exit at the bottom would return the fee itself as profit. Paying the lesser of the par and oracle conversions keeps the bound at the par form, since the oracle branch can then only reduce the payout, and it also addresses the case where an above peg reading has the desk pay out USDC worth more than the dollar it is accounting for, which is value presently lost to every redeemer during a premium rather than to an attacker alone. Such a read would be better made freshness only rather than banded, matching the reasoning already set out in the `cashValue(...)` function, so that a genuinely de-pegged reading selects the par branch instead of taking the desk down. This remains a refinement rather than a substitute: the bound it preserves is still  $(1 + \text{band}) * (1 - \text{fee})$ , so the band comparison itself still needs correcting.

**Status:** Fixed

**Update from the client:** Confirmed to the wei against the live configuration (`band = 50`, `fee = 50`): the truncating comparison admitted up to  $\$1.005099999999999999$ , break-even sits at  $\$1.005025125628140703$ , so the window was real with a ceiling of 0.745 bps. The comparison is now exact (`diff * BPS > maxDeviationBps * WAD`) — the band that binds is the band configured. The nominal edge ( $\$1.005$ ) stays admitted and returns the intended  $\text{fee}^2$  loss (0.25 bps, pinned by test); both invariant sites remain correct as-is because they compare bps against bps.

We verified closure rather than asserting it, with three tests: the band binds **to the wei** on both sides (nominal edge admitted, one wei past it reverts); the maximum admitted price is strictly **below** the round-trip break-even — a property derived from band and fee rather than a sampled case, so no admitted price can pay; and a sweep of round trips at five admitted prices all return strictly less than deposited. We also mapped every consumer of the stable price to confirm no other route credits a premium, and that turned up one of our own: `previewMint` read the oracle **raw**, quoting mints the desk refuses (at  $\$1.02$  it promised 1020 uWatt per 1000 USDC while the deposit reverted). We routed it through the same `_stablePriceChecked` in [4193f23](#) — it's a view and extracts nothing, but a quote should never be greener than the desk, and `previewRedeem` already held that property.

We take your framing seriously: that guard was added specifically to make this round trip unprofitable, and it did not bind at the value it names. We're separately reviewing whether the admissible band should be derived from the exact break-even ( $\text{fee} / (1 - \text{fee})$ ) so the invariant holds with margin instead of resting entirely on the  $\text{fee}^2$  term. We also noted your warning on the redemption leg: paying at par is what bounds the round trip to  $(1 + \text{band})(1 - \text{fee})$ , and converting at the oracle price outright would widen that bound rather than close it — if we make that leg oracle-aware it will be as the lesser of the two conversions, freshness-only. Fixed in [ff70abc](#) (`_branch fix/audit-round-1_`)

**Update from the Nethermind security team:** The band comparison is now exact and the deposit credit is capped at `min(oracle, \$1)`, so the single-asset round trip loses the full fee at every admitted price. The redemption leg is unchanged, however, and still converts to USDC at par with no oracle read, so entering through any other registered deposit asset re-opens the window. Against the fixed code, with a second stable at par and USDC printing  $\$1.0051$ , a 10,000 deposit mints 10,000 uWatt and redeems for 9,950 USDC. That is a loss in token terms but worth  $\$10,000.745$  in market terms, the same 0.745 bps the original window returned, and buying the deposit stable with USDC at market first still leaves the round trip up 0.74 USDC.

**Update from the client (re-review):** Confirmed, and it does not apply to the live configuration: there is a single registered deposit asset (USDC), so there is no second stable to enter through. The residual is a precondition on ever adding one, and we've recorded it in the risk-posture runbook: a second deposit stable is not registered until the redemption leg pays the lesser of the par and oracle conversions (freshness-only), which is the change that closes the market-terms window you describe. Making that leg oracle-aware is the R2 item; until a second stable exists there is nothing to exploit.

## 6.8 [Low] The energy issuance path in the SunoLogicReserve contract does not enforce the solar concentration ceiling

**File(s):** [contracts/core/SunoLogicReserve.sol](#), [contracts/UWattIssuer.sol](#), [contracts/UWattIssuer.sol](#)

**Description:** The reserve backing uWatt holds cash, bonds and tokenized solar projects, and solarCapBps bounds how concentrated it may become in solar. The code treats this as its most important risk control, stating in the `acquireProjectTokens(...)` function of the `UWattIssuer` contract that "the stress model showed capping this matters MORE than C\* itself", and the collateral sizing rule in `setRiskPosture(...)` is derived from the assumption that solar never exceeds solarCapBps of the reserve.

Two paths increase the booked energy value. The first is the `acquireProjectTokens(...)` function, where custody pays USDC for project tokens, and it enforces the ceiling before booking anything:

```
1 uint256 energyAfter = energy + usdWAD;
2 if (energyAfter * BPS > uint256(solarCapBps) * reserveValue()) {
3     revert SolarCapExceeded(energyAfter, reserveValue());
4 }
```

The second is the `swapProject(...)` function of the `SunoLogicReserve` contract, which pulls project token holders' pWatts into the reserve and mints uWatt against them. Its own documentation calls this the energy mint path of uWatt and the only issuance path the contract implements. The per holder `_swapHolder(...)` function mints and books with no reference to the resulting composition:

```
1 if (toHolder > 0) ctx.uWatt.mint(holder, toHolder);
2 if (toProtocol > 0) ctx.uWatt.mint(stakingFeeRecipient, toProtocol);
3 // @audit-issue The energy value is booked with no check on the resulting solar
4 // concentration, unlike the issuer's acquisition path.
5 r.usd = reporter.recordAcquisition(projectToken, received); // book per holder, same step
```

No risk posture parameter is referenced anywhere in the `SunoLogicReserve` contract, and the `recordAcquisition(...)` function of the `EnergyAssetsReporter` contract, which both paths funnel through, does not check it either. The only bound is the `minterAllowed` budget the uWatt token enforces per minter, which caps how much uWatt this contract may ever mint but says nothing about composition. The `UWattIssuer` contract meanwhile states the opposite of what the code does, naming both paths as covered:

```
1 // @audit-issue The solar ceiling is not enforced in the reserve's swapProject.
2 // A3 (capped-peg): _checkNonEnergyCap removed – the deposit-side non-energy
3 // cap is retired; the constraint inverted to the solar CEILING enforced in
4 // acquireProjectTokens (and the reserve's swapProject).
```

Starting from a reserve sitting exactly at the 80% genesis ceiling, one batch of swaps carries it past the bound:

```
1 before    solar $108 = 80.0% of R,  cash $27,  R $135,  S 100,    C 1.35
2 swapProject books $100 of auditor priced solar, minting 100 / 1.35 = 74.07 uWatt
3 after     solar $208 = 88.5% of R,  cash $27,  R $235,  S 174.07,  C 1.35
4 40% shock on solar: loss $83.20 -> R $151.80 -> C 0.872
```

The swap is value neutral in the sense the contract claims, since the collateral ratio is unchanged at 1.35 and the uWatt minted is worth precisely the energy booked. What changes is the composition: solar rises from 80% to 88.5% of the reserve, the liquid tranche falls to 11.5% against a declared `liquidFloorBps` of 20%, and the position after the shock the protocol declares absorbable deteriorates from a collateral ratio of 0.918 to 0.872. Repeating the operation drives concentration toward 100%, where the post shock ratio converges to  $C * (1 - \text{designShock})$ , or 0.81 at the genesis dials. The same operation through `acquireProjectTokens(...)` would revert with `SolarCapExceeded`.

The consequence is that the control the code identifies as its most important one is not enforced on the path that issues uWatt against energy, so the collateral sizing rule rests on a premise the protocol does not maintain, and a solar write down within the declared tolerance leaves holders redeeming at  $\min(\$1, C)$  below a dollar by a wider margin than the configured posture implies. The severity is limited because `swapProject(...)` is restricted to `SWAP_OPERATOR_ROLE` and the project must be registered in the `PRODUCTION` stage, so crossing the ceiling requires a trusted operator action and the loss only materialises if an adverse revaluation follows. It is worth noting that the breach arises from the intended use of the function rather than from misuse, and that it is distinct from the passive drift the code already accepts as unrevokable, since this path actively adds solar exposure and mints against it.

**Recommendation(s):** Consider enforcing the reserve composition constraint on every path that increases the booked energy value, not only on the issuer's acquisition path. Note that the guard cannot be copied verbatim, because the acquisition exchanges cash for solar and leaves the reserve total unchanged whereas the swap adds solar and mints new supply, so the reserve total grows by the booked amount and the comparison must be made against the post operation reserve. The declared liquid tranche is worth revisiting alongside it, as an independent constraint rather than a consequence of the first: the `liquidValue()` function counts cash and `PERMISSIONLESS` bonds only, so a non solar remainder held entirely in gated bonds satisfies the ceiling while still leaving the liquid tranche under `liquidFloorBps`, and that tranche is currently read only when cash actively leaves custody.

**Status:** Fixed

**Update from the client:** Confirmed — `swapProject`, the energy `_mint_path`, booked energy and minted uWatt with no check on the resulting solar concentration, unlike the issuer's `acquireProjectTokens`. One batch at the 80% genesis ceiling took solar to 88.5% of the reserve, deepening the post-shock C from 0.918 to 0.872; repeated, it converges on  $C * (1 - \text{designShock})$ .

**Fix:** swapProject now enforces  $E \leq \text{solarCap} * R$  once after the batch, against the **post-operation** reserve — the batch both books energy and mints uWatt, so the reserve grows by the booked amount, and the comparison is made after that growth. solarCapBps and reserveValue() are read from the issuer (the single owner of the risk posture) via the registry, fail-closed if the issuer is unregistered. Red→green: a batch that would carry solar past the ceiling reverts SolarCapExceeded, and a swap that leaves solar under the ceiling proceeds.

**On the related note about the liquid tranche:** we agree liquidValue() counts cash and PERMISSIONLESS bonds only and is read only when cash leaves custody, so a non-solar remainder held entirely in gated bonds can satisfy the solar ceiling while leaving the liquid tranche under liquidFloorBps. We're treating that as a separate item — enforcing liquidFloorBps as an independent composition constraint (not only an outflow guard) — and will address it in a follow-up rather than fold it into this fix. Fixed in [83b135b](#) (\_branch fix/audit-round-1\_)

**Follow-up — the liquid sleeve as a standing constraint.** As promised, we revisited liquidFloorBps beyond the outflow guard. The sleeve was read only when cash left custody for a bond, so a non-solar remainder held in gated bonds could sit under the floor unchecked.

- acquireProjectTokens now applies the same \_checkBufferAfter that deployCash and redeemBond use: buying solar spends custody cash, so with a gated bond occupying part of the non-solar remainder it could drop the liquid tranche below the floor while staying under the solar cap. (Absent gated bonds the solar cap already implies  $\text{liquid} \geq \text{floor}$ , since  $\text{solarCap} + \text{liquidFloor} \leq \text{BPS}$ .);
- swapProject (the energy mint path) is deliberately left **monitor-watched** rather than hard-reverting: it grows the reserve without moving cash, and it is the genesis onboarding path — Fase 5 builds the reserve solar-first, before user deposits bring cash, so a hard floor check would revert every bootstrap swap. We treat this as monitored composition drift, consistent with how the desk already handles passive drift (redemptions draining cash, revaluations), which the code notes "can't be reverted here — the monitor flags them.";

Red→green: acquiring solar past the floor now reverts RedemptionBufferBreached (with a gated bond present); a smaller acquire under the floor proceeds. Fixed in [af317a6](#) (\_branch fix/audit-round-1\_)

**Update from the Nethermind Security Team:** The solar ceiling is now enforced on the energy mint path against the post-operation reserve, which resolves the reported issue, but the reasoning recorded alongside it does not hold. The code comment in swapProject(...) justifies leaving liquidFloorBps unenforced on the grounds that Fase 5 builds the reserve solar-first, before user deposits bring cash, and a hard floor check would revert every bootstrap swap. Since reserveValue() sums energy, bonds and cash, a solar-first reserve makes energyAfter equal to reserveAfter, so the solar cap check added in the same function already reverts that swap. Any swap now requires the reserve to already hold enough non-energy value to keep total booked energy under solarCapBps, so the described bootstrap flow is not one the contract accepts. We raise this so the genesis sequence and the reasoning recorded in the comment can be reviewed against the enforced behaviour.

**Update from the client (re-review):** Agreed, the comment's reason was wrong — a solar-first swap reverts on the solar cap, not on any floor check; the floor is monitor-watched only because swapProject moves no cash (composition drift). Comment rewritten. The actual bootstrap: genesis sets an \_onboarding posture\_ (solarCapBps = 100%, liquidFloorBps = 0) **before** the first swapProject, making the cap check vacuous, and applies the production posture after — the sequence the contract accepts, exercised in our fork rehearsal. That leaves a deliberate \_passive\_ breach of the production cap (98% energy vs 60%; \$100k seed vs \$5.0M energy), accepted because the ceiling gates active paths only. Consequence for your [Medium]: the sizing rule assumes  $E \leq \text{cap} * R$ , so real shock coverage at genesis is 9.3% (98% energy, 15% shock → C 0.938), not 15% — kept by decision, with these numbers on record. Fixed in [411a83e](#).

## 6.9 [Low] Users can only have their project tokens swapped once causing subsequent tokens to be stuck

**File(s):** [contracts/core/SunoLogicReserve.sol](#)

**Description:** The `swapProject(...)` function from the `SunoLogicReserve` contract allows swapping pWatt project tokens for uWatt tokens. This process delegates to the internal `_swapHolder(...)` function to execute the token exchange for each individual holder.

Whenever a user has their pWatt tokens swapped to uWatt tokens for the first time, the `swapProject(...)` flow sets the `swapped[projectToken][holder]` flag to true. However, if at any point in time after this initial swap, the user receives extra pWatt tokens from the project, they will be unable to swap them.

When the internal `_swapHolder(...)` function is called for subsequent swaps, it checks this flag. Because the flag was set to true during the previous operation, the function returns early without performing any token exchange.

```

1  function _swapHolder(
2      address projectToken,
3      address holder,
4      SwapCtx memory ctx,
5      IEnergyAssetsReporter reporter
6  ) internal returns (HolderResult memory r) {
7      // @audit-issue The function returns early if the swapped flag is already true.
8      if (holder == address(this) || swapped[projectToken][holder]) return r;
9      // ...
10 }
11

```

As a result, users can't have any future swaps for the same project token. This causes any additional pWatt tokens they receive to become stuck in their accounts.

**Recommendation(s):** Consider revisiting the swap logic to ensure that users who acquire additional project tokens after their initial swap are still able to exchange them.

**Status:** Fixed

**Update from the client:** The `swapped[project][holder]` marker no longer gates `_swapHolder`; it is still written, as an informational "swapped at least once" flag our off-chain reconciliation reads. Safety argument for allowing re-swaps: idempotency of retries and overlapping batches is already provided by the zero-balance skip, and each swap mints exactly the auditor-priced value it books in the same step (value-neutral by construction), so a repeat swap cannot create unbacked supply. The prior unit test that encoded the stranding as intended behavior was corrected as part of this commit. Red→green tests: a second distribution to an already-swapped holder is now swappable; a retry against a zero balance still reverts `NothingSwapped`. Fixed in [0f83bb7](#) (`_branch fix/audit-round-1`)

## 6.10 [Info] Atomic acquisitions and disposals incorrectly refresh the aggregate energy valuation timestamp

**File(s):** `contracts/EnergyAssetsReporter.sol`

**Description:** The EnergyAssetsReporter contract maintains the aggregate valuation of the protocol's energy assets. Functions like `recordAcquisition(...)` and `recordDisposal(...)` are used to adjust the aggregate reserve value whenever assets are added to or removed from the reserve. The `updatedAt` variable stores the timestamp of the last valuation and serves as a freshness check for the UWattPriceOracle and UWattIssuer contracts.

Inside both the `recordAcquisition(...)` and `recordDisposal(...)` functions, the `updatedAt` variable is updated to `block.timestamp`. Since this timestamp is used to validate the entire reserve, updating it during an acquisition or disposal extends the validity of the aggregate Net Asset Value (NAV) without a comprehensive reevaluation of the underlying assets.

```
1 function recordAcquisition(...) external {
2     // ...
3     // @audit-issue This extends the validity of the aggregate NAV.
4     updatedAt = uint64(block.timestamp);
5 }
6
```

Similarly, the `recordDisposal(...)` function mimics the same behavior:

```
1 function recordDisposal(...) external {
2     // ...
3     // @audit-issue This extends the validity of the aggregate NAV.
4     updatedAt = uint64(block.timestamp);
5 }
6
```

By executing a minimal swap or acquisition, a swap operator can refresh the timestamp for the entire reserve. This bypasses the `maxOracleStaleness` circuit breaker, allowing the protocol to continue minting and redeeming `uwatt` tokens based on potentially stale aggregate valuations. This breaks the protocol's expected capped-peg mechanics and potentially enables value extraction against outdated prices, which could drain the reserve.

**Recommendation(s):** Consider removing the `updatedAt` timestamp updates from the `recordAcquisition(...)` and `recordDisposal(...)` functions. The aggregate timestamp should track the last time the entire reserve was valued via the main reporting functionalities.

**Status:** Fixed

**Update from the client:** Fixed as recommended: `recordAcquisition` and `recordDisposal` no longer write `updatedAt`. They are value-neutral bookings (matched 1:1 by cash or burn in the same transaction), not revaluations, so a minimal swap can no longer extend the whole NAV's validity past `maxOracleStaleness`. The staleness clock now tracks `report()/reportUnchecked()` only. The genesis guard in `report()` (`last == 0 && updatedAt != 0 → MustUseUnchecked`) keeps identical semantics. Red→green test: bookings move value but not `updatedAt`; only a real report refreshes the clock. Fixed in [da9ad10](#) (`_branch fix/audit-round-1_`)

## 6.11 [Info] Incorrect event emission in the swap function causes a permanent denial of service via the reconciliation monitor

**File(s):** [contracts/core/SunoLogicReserve.sol](#)

**Description:** The `pWattSwap(...)` function from the `SunoLogicReserve` contract allows the swap operator to exchange `uWatt` tokens for multiple `pWatt` project tokens in a single batch. The function calculates the total amount of `uWatt` to burn (`uWattIn`) for the entire batch up front.

However, when emitting the `PWattsSwapped` event inside the loop that transfers the project tokens, the function incorrectly logs the total aggregate `uWattIn` amount for each individual project token in the batch.

```

1  function pWattSwap(...) external {
2      // ...
3      for (uint256 i = 0; i < projectTokens.length; i++) {
4          // ...
5          // @audit-issue The event logs the aggregate uWattIn for each project token.
6          emit PWattsSwapped(receiver, projectTokens[i], uWattIn, pWattAmounts[i]);
7      }
8      // ...
9  }
10

```

If an operator swaps five different project tokens in one batch, the event is emitted five times, each time logging the total batch burn amount. The event emission overstates the amount burned. If the protocol's off-chain monitoring systems rely on the accuracy of the burned `uWattIn` value in order to trigger the `pauseFromMonitor(...)` function on the `UWattIssuer` contract, the `MONITOR_ROLE` may erroneously call `pauseFromMonitor(...)` too early. The system will require manual intervention from governance to unpaue it.

**Recommendation(s):** Consider moving the event emission outside the loop to log the aggregate swap for the entire batch. Alternatively, consider calculating and emitting the proportional `uWatt` burned for each specific project token inside the loop.

**Status:** Fixed

**Update from the client:** We took the per-token option: each `PWattsSwapped` now logs its token's **share** of the burn (floor over that token's auditor-priced USD).  $\sum \text{shares} \leq \text{uWattIn}$  (the total rounds up) by less than `n wei`; the exact per-token USD remains available in the reporter's `DisposalRecorded`. The event signature is unchanged, and we verified our current off-chain consumers aggregate `pWattsOut` rather than `uWattsBurned`, so no monitor change is required. Red→green test: a two-token batch at different auditor prices emits exact per-token shares summing to the burn within rounding dust. Fixed in [37d96b0](#) (`_branch fix/audit-round-1`)

## 6.12 [Info] The BLOCKER\_ROLE can freeze all uWatt issuance and redemption, which pausing is supposed to gate

**File(s):** `contracts/ERC20UWattUpgradeable.sol`, `contracts/ERC20UWattUpgradeable.sol`, `contracts/crosschain/XCUWatt.sol`

**Description:** The ERC20UWattUpgradeable contract separates its privileged roles by blast radius. The PAUSER\_ROLE holds the wide lever, since the `pause(...)` function freezes every transfer at once, while the BLOCKER\_ROLE is meant to be the narrow one, denying individual accounts while the rest of the protocol keeps running.

The transfer hook rejects three addresses, not two:

```

1  if (blocked[_from]) revert Blocked(_from);
2  if (blocked[_to]) revert Blocked(_to);
3  // @audit-issue The CALLER is rejected too, so blocking a contract disables
4  // everything that contract drives.
5  if (blocked[_msgSender()]) revert Blocked(_msgSender());

```

Because the caller is checked, blocking one address disables the module behind it. The UWattIssuer proxy is both the minter and the burner, so blocking it stops the `depositAndMint(...)` and `redeemUWatt(...)` functions, and the same applies to the `SunoLogicReserve` and `YieldController` contracts. The `mint(...)` and `burn(...)` functions carry the check directly as well, through the `notBlocked(_msgSender())` modifier.

The `addToBlockedList(...)` function also accepts the zero address:

```

1  function addToBlockedList(
2      address account
3  )
4  // @audit-issue No notZero, though the modifier exists in this contract and is
5  // applied on setUpgrader(...) and transferOwnership(...).
6  external onlyRole(BLOCKER_ROLE) {

```

A mint reaches the hook with `_from` as the zero address and a burn with `_to` as the zero address, so blocking it rejects every mint and every burn at once. That is a complete halt of issuance and redemption from a single call, with no PAUSER\_ROLE involved. Two things in the codebase already treat that as impossible. This contract's own hook documentation states that "address(0) is never blocked, so mint/burn keep working", which nothing enforces. And XCUWatt, introduced in the code as the "Same compliance shell as uWatt v2", is defended against this on three fronts where the canonical token is defended on none: its setter carries `notZero`, its hook skips both checks when `_from` is the zero address so mints are exempt, and it never inspects `_msgSender()`. Its documentation records why, noting that blocking the zero address would otherwise hand the role "a protocol-wide kill switch over the exit paths, which is pausing's job and a different role's decision". The hazard was recognised and closed on one of the two tokens and left open on the other.

The impact is a denial of service on issuance and redemption available to a role that is not meant to have it. It is reversible through the `removeFromBlockedList(...)` function and moves no value, so this rests on the role model rather than on direct loss, and the exposure is larger if the BLOCKER\_ROLE is held at a lower trust level than the PAUSER\_ROLE.

**Recommendation(s):** Consider applying the contract's existing `notZero` modifier to the setter, matching both XCUWatt and the invariant this contract's own hook documentation already assumes. Separately, consider whether the hook should inspect `_msgSender()` at all: denying an account is a statement about the counterparties to a movement, whereas rejecting the caller extends it to every contract acting on a user's behalf. If that is intentional for the compliance model, consider exempting the protocol's own contracts, or documenting that the BLOCKER\_ROLE must be held at the same trust level as the PAUSER\_ROLE.

**Status:** Fixed

**Update from the client:** Confirmed on both fronts, and closed on **both** uWatt v2 and SunoProjectToken (the pWatts) — as you note, XCUWatt already carried these defenses and the hazard was left open on the other two tokens.

**address(0) was blockable:** a mint is a transfer from `address(0)` and a burn one to `address(0)`, so blocking the zero address reverted `_every_` mint and burn from a single BLOCKER\_ROLE call — the protocol-wide halt pausing is meant to gate. `addToBlockedList` now carries `notZero` (uWatt v2, via the existing modifier) / reverts `ZeroAddress (SunoProjectToken)`, matching XCUWatt and enforcing the invariant both hooks already `_assumed_` ("address(0) is never blocked") but never checked.

**The caller was rejected, not just the counterparties:** the hook rejected `blocked[_msgSender()]`, and on uWatt v2 mint/ burn carried `notBlocked(_msgSender())` directly — so blocking a protocol module (the UWattIssuer proxy is both minter and burner; the reserve and YieldController drive movements) disabled everything it drives. The caller checks are removed on both tokens. The `from/ to` checks already cover both endpoints of the value movement, so a blocked holder still cannot send, receive, or burn, and a mint to a blocked holder is still rejected ( `notBlocked(to)` on mint + the hook's `blocked[_to]`). The approve family keeps its checks — that is standard blocklist behavior, not an issuance path.

Red→green: `address(0)` is unblockable on both tokens; blocking the minter module no longer halts issuance to clean holders; mint-to-blocked still reverts; a blocked holder still cannot transfer. 263 tests. Fixed in [Oed481f](#) (`_branch fix/audit-round-1_`)

**Update from the Nethermind Security Team:** Dropping the caller check narrows the blast radius without removing it, since several protocol addresses are themselves an endpoint of the movement they drive. Blocking the UWattIssuer proxy still reverts every `redeemUWatt(...)` call, blocking the SunoLogicReserve contract reverts `pWattSwap(...)`, blocking the SunoStakingVault contract freezes staker exits, blocking the SunoLockbox contract freezes bridge exits, blocking the `stakingFeeRecipient` address reverts `swapProject(...)` on the protocol fee mint, and blocking the YieldController contract reverts `distributeEnergyPayment(...)`. We read this as an accepted operational constraint rather than a defect, though it does mean the trust-level note in the recommendation still applies. One consequence of the change does need attention. The removed `blocked[_msgSender()]` check was the only place the spender was inspected, and the approve

family remains gated on `notBlocked(spender)`. A blocked account holding a pre-existing allowance can therefore call `transferFrom(...)` to move an unblocked holder's `uWatt` to an address of its choosing, while the holder cannot revoke that allowance because `approve(...)` and `decreaseAllowance(...)` revert `Blocked`. The blocklist ends up denying the victim rather than the blocked account, which inverts the intent of the mechanism.

#### Update from the client (re-review):

Confirmed. The re-review is right on both points, and the second one is a regression our fix introduced.

**On the spender inversion.** Dropping `blocked[_msgSender()]` from the hook removed the only place the spender was inspected, while `approve / increaseAllowance / decreaseAllowance` stayed gated on `notBlocked(spender)`. Net effect, exactly as you describe: a blocked account holding a pre-existing allowance could `transferFrom(...)` an unblocked holder's `uWatt` (from/to clean, spender unseen), and the holder could not revoke because `approve(...)` and `decreaseAllowance(...)` reverted `Blocked`. The blocklist denied the victim rather than the blocked account.

Fixed on `uWatt v2` in two places:

- `transferFrom(...)` now carries `notBlocked(_msgSender())` — the spender is inspected here and only here. This does not re-open the module-blocking DoS the original fix closed: every protocol module that calls `transferFrom` as spender is also an endpoint of that same movement (the issuer is to in redeem — it pulls via `safeTransferFrom` then burns its own balance; the vault is to on stake; the lockbox is to on bridge), so blocking any of them was already covered by the from/to checks. No new blast radius beyond the set you enumerated;
- `decreaseAllowance(...)` drops `notBlocked(spender)` (the holder check stays). Reducing an allowance is monotone-down, so a holder can always shrink or revoke what a now-blocked spender could draw. `approve` and `increaseAllowance` remain gated on the spender: exposure to a blocked account can never be raised, only lowered. This restores the holder's control rather than leaving the allowance merely inert;

Red→green: a blocked spender's `transferFrom` on a pre-existing allowance reverts `Blocked(spender)` with nothing moved; the holder can `decreaseAllowance` to zero while `approve / increaseAllowance` toward the blocked spender still revert; an unblocked spender's `transferFrom` is unchanged. 270 tests.

**On the endpoint blast radius.** Accepted as an operational constraint, as you read it. We took the trust-level note as a real gap on our side: `BLOCKER_ROLE` was absent from our custody map. It is now recorded at the same trust level as `PAUSER_ROLE`, with the explicit list of protocol addresses whose own flows it can halt (issuer on redeem, reserve on `pWattSwap`, vault, lockbox, `YieldController`, `stakingFeeRecipient`), so it is never assigned to a hotter key than pause.

`XCUWatt` and `SunoProjectToken` are unchanged: neither gates `approve` on the spender, so a holder can always revoke there — no inversion, only the standard ERC20-blocklist window where a spender may draw until revoked, which is the reference behavior `XCUWatt` already had. Happy to harmonize the spender check onto them too if you'd prefer symmetry.

Fixed in [b2e5466](#) (branch `fix/audit-round-1`).

**Update from the Nethermind Security Team:** Fixed

## 7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

### Remarks about Suno' documentation

Suno's development team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and an in-depth understanding of the project's technical aspects.

The documentation included a dedicated docs folder which presents information about the protocol's architecture, economic model, invariants, and threat model. The `Invariants.md` file presents the most important economic and architectural invariants.

A standout was the `Operational Manual` which documents the duties of trusted roles in the protocol, explains when and how each role will perform their duties, as well as the expected values that will be set by governance for the admin-controlled parameters throughout the protocol.

## 8 Test Suite Evaluation

### 8.1 Tests Output

#### Remarks about the compiler warning

The compiler linearization warning is harmless, it won't cause issues with the deployment.

Capped-peg Fase B – vault vesting + YieldController SunoStakingVault (Synthetix vesting)  
Warning: Potentially unsafe deployment of contracts/core/SunoStakingVault.sol:SunoStakingVault  
contracts/core/SunoStakingVault.sol:89: Incorrect order of parent initializer calls.  
- Found initializer calls to parent contracts in the following order: ERC4626Upgradeable, ERC20Upgradeable,  
↳ ReentrancyGuardUpgradeable  
- Expected: ERC20Upgradeable, ERC4626Upgradeable, ReentrancyGuardUpgradeable  
Call parent initializers in linearized order

streams a reward linearly: pps rises over the window, not in a step (725ms)  
anti-JIT: a just-in-time depositor cannot capture unvested rewards (59ms)  
compliance: uWatt pause freezes shares; blocked holder can't move out (49ms)  
notifyReward is REWARDER-only  
YieldController (= clamp buffer controller)  
distributes toward C\* and streams to the vault; retains the rest as buffer (45ms)  
drives C toward C\*: after a payment the composed C moves up (rebuild)  
at/above C\*: distributes ~all of Y (steady state, holds the buffer)  
fails closed on a stale energy reporter  
distributeEnergyPayment is ENERGY\_PAYER-only and rejects zero  
YieldController – RAMP (C\_y = yieldStartWad; default 1.0 = linear-from-par)  
IN-BAND (1 < C < C\*): yield flows AND C still rises – the fix for the dead zone (43ms)  
AT or BELOW par: retains everything (hard floor, independent of C\_y)  
at C\*: 1/C\* (holds the target, keeps feeding the buffer)  
setYieldStart: bounds [1.0, C\*), admin-only, and rebuild-first behavior (66ms)  
windfall (C C\*): caps at 1 – all of Y distributed

SunoStakingVault – first-depositor & empty-vault reward capture (scan 5, H2/H3)  
Warning: Potentially unsafe deployment of contracts/core/SunoStakingVault.sol:SunoStakingVault  
contracts/core/SunoStakingVault.sol:89: Incorrect order of parent initializer calls.  
- Found initializer calls to parent contracts in the following order: ERC4626Upgradeable, ERC20Upgradeable,  
↳ ReentrancyGuardUpgradeable  
- Expected: ERC20Upgradeable, ERC4626Upgradeable, ReentrancyGuardUpgradeable  
Call parent initializers in linearized order

blocks any deposit into an unseeded vault – closing the front-run window (115ms)  
Warning: Potentially unsafe deployment of contracts/core/SunoStakingVault.sol:SunoStakingVault  
contracts/core/SunoStakingVault.sol:89: Incorrect order of parent initializer calls.  
- Found initializer calls to parent contracts in the following order: ERC4626Upgradeable, ERC20Upgradeable,  
↳ ReentrancyGuardUpgradeable  
- Expected: ERC20Upgradeable, ERC4626Upgradeable, ReentrancyGuardUpgradeable  
Call parent initializers in linearized order

bootstrap is governance-only, one-shot, and opens the vault (144ms)  
Warning: Potentially unsafe deployment of contracts/core/SunoStakingVault.sol:SunoStakingVault  
contracts/core/SunoStakingVault.sol:89: Incorrect order of parent initializer calls.  
- Found initializer calls to parent contracts in the following order: ERC4626Upgradeable, ERC20Upgradeable,  
↳ ReentrancyGuardUpgradeable  
- Expected: ERC20Upgradeable, ERC4626Upgradeable, ReentrancyGuardUpgradeable  
Call parent initializers in linearized order

the donation inflation attack no longer steals: victim keeps their value, attacker loses the donation (173ms)  
Warning: Potentially unsafe deployment of contracts/core/SunoStakingVault.sol:SunoStakingVault  
contracts/core/SunoStakingVault.sol:89: Incorrect order of parent initializer calls.  
- Found initializer calls to parent contracts in the following order: ERC4626Upgradeable, ERC20Upgradeable,  
↳ ReentrancyGuardUpgradeable  
- Expected: ERC20Upgradeable, ERC4626Upgradeable, ReentrancyGuardUpgradeable  
Call parent initializers in linearized order

notifyReward reverts on an unseeded (zero-supply) vault (125ms)

UWattIssuer \* YieldController - C\* > C\_y guarded from both sides (scan 5, F21)  
 setRiskPosture cannot lower C\* to/below the controller's yieldStart once wired  
 the cross-check is skipped while no controller is wired (backwards compatible)

ChainlinkStableOracle  
 normalizes the 8-decimal feed answer to WAD and passes the feed timestamp through (65ms)  
 handles feeds with more than 18 decimals by scaling down  
 fails closed: unregistered asset and non-positive answers revert  
 setFeed is owner-gated and caches the feed decimals

UWattIssuer.setStableOracle (rotation mock -> Chainlink)  
 rotates the oracle and deposits validate against the Chainlink price (186ms)  
 a de-pegged Chainlink answer blocks deposits (band 50 bps)  
 rotation is role-gated and rejects the zero address

ChainlinkStableOracle (mainnet fork)  
 - reads the real USDC/USD feed inside the de-peg band

contract size - the issuer has no room  
 UWattIssuer 23446 bytes · 1130 to spare of 24576  
 reports UWattIssuer's headroom, and fails with an explanation when it runs out

Cross-chain bridges (LayerZero + Axelar) - e2e local

LayerZero adapter  
 bridges L1 -> L2 (burn on source, mint on destination) and back, conserving supply (239ms)  
 rejects delivery from an untrusted peer  
 #28 rejects a zero-sender origin on an unconfigured route (40ms)  
 #29 owner can move the LayerZero delegate; others cannot (41ms)

Axelar adapter (same xERC20, independent limit)  
 bridges L1 -> L2 via Axelar GMP and mints on execute (111ms)  
 rejects execute from an untrusted source address (49ms)

Cross-chain (protocol\_v3) - Lockbox + XCUWatt + OracleL2

SunoLockbox <-> XCUWatt (1:1 home minter)  
 deposit locks canonical and mints xERC20 1:1; withdraw reverses (133ms)  
 rescue cannot touch the canonical

xERC20 rate-limited bridge (EIP-7281)  
 a bridge mints within its limit and reverts beyond it; the lockbox is exempt (67ms)  
 an unauthorized address (no limit) cannot mint

compliance (mirrors uWatt v2)  
 pause freezes lockbox deposit AND bridge mint (52ms)  
 a blocked holder receives mints but cannot transfer out (43ms)  
 cannot blacklist the zero address - it would brick every burn, and every exit with it

SunoPriceOracleL2 (Curve rate + dual staleness + push guards)  
 exchangeRate returns the rate; keeper push is deviation-bounded (51ms)  
 exchangeRate reverts (halts the pool) when the push is stale  
 THROTTLE: a second push before minUpdateInterval reverts TooSoon (39ms)  
 ANCHORED BAND: the multi-push walk reverts once CUMULATIVE move exceeds the band within a window (65ms)  
 ABSOLUTE BOUNDS: setRate and adminSetRate revert outside [minRate, maxRate] (52ms)  
 adminSetRate: skips throttle+band but resets the anchor so the keeper resumes cleanly (57ms)  
 HAPPY PATH: spaced, in-band pushes work and exchangeRate stays fresh (57ms)  
 setGuards: owner retunes guards but cannot brick the live rate (39ms)

CUWattPriceOracle (D3 - on-chain c-uWatt composition)  
 CAPS the uWatt leg at par and composes with pricePerShare (85ms)  
 closes the bridge arbitrage: par mint -> stake -> bridge is value-neutral  
 is transparent below par (impairment passes through 1:1)  
 tracks canonical price moves (both directions)  
 fails closed by construction when the canonical oracle is stale  
 propagates the REAL canonical oracle revert (full-stack: reporter stale) (146ms)  
 rejects zero addresses at construction

Mock fidelity vs mainnet (Sky)  
 sUSDS accrues yield continuously like the SSR (no manual pokes between sessions) (114ms)  
 sUSDS redeem always pays accrued yield (vat solvency stand-in)  
 PSM bounded capacity: an oversized single call reverts (fill()-bounded buffer) (49ms)

## Price oracle stack (interim option A) + contract custody

## EnergyAssetsReporter

first report sets the value; subsequent reports are band-limited (30 bps) (342ms)  
 reportUnchecked is the governance escape hatch; report is role-gated  
 setAssetPrice is auditor-only; recordAcquisition is ACQUIRER-only and bumps value (204ms)  
 #27 a banded report after value is drained to zero is refused (must use unchecked) (59ms)  
 setAssetPrice stamps a timestamp; assetPriceInfo exposes both (staleness gate input) (38ms)  
 recordDisposal mirrors recordAcquisition: ACQUIRER-only, lowers value, guards underflow (130ms)  
 is a UUPS proxy: upgrade keeps value/prices/roles at the SAME address (no oracle-chain redeploy) (127ms)

## UWattPriceOracle staleness dial is bounded

rejects below 1d and above 30d, accepts in range

## UWattPriceOracle (composed)

composes price = (energy + bond + cash) / supply  
 fails closed on stale reporter  
 setIssuer is once-only and owner-gated

## end-to-end with the composed price

capped-peg: a par mint DILUTES the composed price toward \$1 (buffer belongs to the system)  
 capped-peg: desk redemption at \$1 RAISES C (burn removes supply at C >= payout)  
 stage-A operator leg through the Safe contract: cash -> sUSDS keeps the price intact (40ms)

## SkySUSDSAdapter

## identity &amp; wiring

exposes the bond token, mode PERMISSIONLESS and starts non-automated (120ms)  
 owner is governance

## navWAD (valuation at real exit rate)

is 1:1 when rate=1 and tout=0  
 tracks the sUSDS exchange rate (yield accrual)  
 discounts the live PSM tout fee (never assumes 0)

## maxInstantRedeemUSDC

returns the pocket USDC balance

## stage gating (doc §3.9)

reverts execution with NotAutomated in stage A  
 only the issuer can execute once automated (106ms)  
 only governance can flip automated / set issuer

## deployToBond (USDC -&gt; sUSDS, custody to custody)

pulls USDC from custody and delivers shares to custody (134ms)  
 mints fewer shares when the rate has accrued  
 enforces minBondOut (slippage guard)  
 rejects zero amounts

## redeemToUSDC (sUSDS -&gt; USDC)

round-trips back to custody, settled=true, no funds retained (141ms)  
 captures accrued yield on exit (rate up -> more USDC out)  
 with tout > 0: pays the fee and returns USDS dust to custody (45ms)  
 enforces minUsdcOut (slippage guard)  
 rejects zero amount and zero recipient

## Suno energy core (Fase 2, Modelo B)

## SunoProjects + SunoProjectToken

factory wires the token roles: reserve can pull, caller can mint, governor is admin (334ms)  
 registers with ONBOARDING stage and rejects duplicates / zero swapFactor  
 registerProject rejects a non-18-decimal token (46ms)  
 gates project admin behind PROJECT\_ADMIN\_ROLE (the funding-module seam)  
 pWatt transfers are disabled by default, reserve bypasses, admin can enable  
 reservePull is RESERVE\_ROLE-only and always lands on the caller

## SunoProjectToken - uWatt-v2-grade safety (pausable, blocklist, cap, upgradeable)

mint is capped by a per-minter allowance (like uWatt v2)  
 PAUSER can freeze all movement; reservePull/swap revert while paused (38ms)  
 BLOCKER can block an address; blocked holders cannot receive mints  
 factory grants PAUSER + BLOCKER to the tokenAdmin (governance)  
 createProject reverts if the beacon is not set (72ms)  
 destination-is-reserve exception: approve+transferFrom INTO the reserve works with transfers disabled  
 beacon upgrade swaps the implementation for ALL project tokens at once

## swapProject - energy mint at the auditor price, booked atomically

mints pWatts\*assetPrice/price for every holder, fee to staking, and books the energy value (441ms)  
 books and mints against the ACTUAL received amount, not the reported balance (70ms)  
 previewSwap matches the on-chain mint amounts  
 property parity: random pWatts/assetPrice/fee - on-chain == formula, price never moves (697ms)  
 is idempotent: re-running the same batch never double-mints (kills legacy L2) (41ms)  
 skips already-swapped holders inside a mixed batch (overlapping snapshots)  
 only swaps PRODUCTION projects and only via SWAP\_OPERATOR\_ROLE  
 respects the uWatt minterAllowed hard cap (334ms)  
 fails closed when the auditor has not priced the project (NoAssetPrice)  
 fails closed when the asset price is stale (StaleAssetPrice)

fails closed when the composed oracle is stale (reporter aggregate too old)  
 enforces the operator's price bounds (PriceOutOfBounds)  
 reverts PriceOracleNotSet until governance wires the oracle (43ms)  
 cannot book without ACQUIRER\_ROLE on the reporter (atomic booking is enforced)  
 pWattSwap – uWatt burn derived from auditor value, booked atomically  
 burns ceil(value/price) uWatt, returns pWatts, and books the energy value DOWN (60ms)  
 enforces the operator's burn bound (SlippageExceeded)  
 fails closed on unpriced/stale asset prices and rejects value-less no-ops (43ms)  
 validates arrays and role  
 emergency controls  
 monitor can pause (not unpause); pause blocks both swap directions  
 governor can unpause and operations resume  
 price-oracle rotation is admin-only and requires the pause (like the issuer)

SunoRegistry

- grants governor the roles and starts empty (76ms)
- sets and resolves an address (governor only)
- getOrRevert reverts on an unset key
- rejects the zero address and non-governor writers
- batch-wires several modules in one tx

UWattIssuer

- depositAndMint (capped-peg: mint at PAR + C-floor, \$3.2/A2)
  - mints at PAR (\$1 = 1 uWatt) and routes funds to custody (264ms)
  - enforces minUWattOut (in par units now)
  - rejects a de-pegged stable (band 50 bps)
  - fails closed on a stale ENERGY reporter (the mint C-floor reads it)
  - C-floor guard: blocks minting into an UNDER-collateralized system ( $C < cFloor$ ) (271ms)
  - stablecoin feed staleness is a SEPARATE, tighter bound (48h) than energy (10d) (46ms)
  - par mint is allowed from a healthy (over-collateralized) state – no desk band trips (251ms)
  - A3: the legacy non-energy cap is RETIRED – large cash deposits succeed (strengthen the peg)
  - enforces the per-asset cash cap (depositCap, \$3.1)
- depositAndMint – RESCUE MINT (option C: mint =  $\min(\$1, C)$ , cFloorWad = rescue floor)
  - prices sub-par at  $\min(\$1, C)$  and stays C-neutral (never lowers C) (290ms)
  - still blocks minting when C is below the rescue floor ( $C < cFloorWad$ )
  - above par: still mints at exactly \$1 even with the floor lowered (242ms)
  - with the floor at 1.0 (default) sub-par minting stays closed (behaves like par-only)
- redeemUWatt (capped-peg desk:  $\min(\$1, C)$  fee, \$3.5/A2)
  - redeems at PAR minus fee when over-collateralized ( $C > 1 \rightarrow$  capped at \$1) (265ms)
  - floats to NAV (below \$1) when UNDER-collateralized – honest, pro-rata
  - blocks mint->redeem in the same block (anti round-trip, \$13.5)
  - enforces the daily redemption limit (1% of supply, \$13.4) (280ms)
  - auto-funds the desk by redeeming sUSDS when custody cash is short (\$12.3) (38ms)
  - reverts with InsufficientDeskLiquidity when there is no cash and no automated adapter
  - #15 auto-funding skips a disabled bond instead of unwinding it
  - desk pause blocks redemptions; global pause blocks both flows
- treasury cycle (\$3.5)
  - deployCash routes custody USDC into sUSDS via the adapter (stage B)
  - redeemBond brings sUSDS back to USDC in custody, settled=true
  - withdrawForReserveOps: allowlist + redemption buffer enforced (\$3.5, \$3.8) (64ms)
  - liquidityFloor: the declared liquid sleeve binds operator outflows (\$3.5) (41ms)
  - treasury functions are role-gated
- NAV views (\$3.1)
  - cash/bond/reserve/ratio derive from custody balances (44ms)
- desk params (\$13.4/\$13.5)
  - rejects params breaking the anti-sandwich invariant fee > jump band
  - admin functions are role-gated
- acquireProjectTokens (atomic cash → energy, no window)
  - pays USDC from custody, pulls pWatts to dest, books energy – value-neutral, no NAV drop (272ms)
  - works on a transfer-RESTRICTED real pWatt when dest holds RESERVE\_ROLE (destination exception) (89ms)
  - refuses to pay custody USDC when dest is the seller (self-transfer books energy for nothing) (39ms)
  - reverts on unpriced asset, slippage, zero, and non-TREASURY caller (276ms)
  - SOLAR CEILING (A3): blocks acquiring more solar above the cap (self-regulating redeploy) (40ms)
  - reverts StaleOracle when the per-project asset price is older than maxOracleStaleness (61ms)
- risk posture (capped-peg A1: dynamic params + sizing-rule invariant)
  - genesis defaults: concentrated posture, consistent by the sizing rule
  - stage transitions move as ONE atomic act and pass the invariant
  - REJECTS an unsafe combination: the sizing rule is enforced by construction
  - rejects malformed postures: infeasible sleeves, cFloor above C\*, bounds
  - is governance-only (DEFAULT\_ADMIN – timelock in prod)

setCustody (Safe rotation runbook)  
rotates custody ONLY while paused, admin-only, non-zero (323ms)  
after rotation + fund move, accounting follows the new custody unchanged (43ms)  
staleness dials are bounded (\$3.8)  
setMaxOracleStaleness: rejects below 1d and above 30d, emits on success  
setMaxStableStaleness: rejects below 1h and above 7d

UWattIssuer – bond valuation is decimal-independent  
caches the bond's real decimals at registration  
values a 6-decimal bond at face, not 1e12 short  
carries the same unit into liquidValue and reserveValue  
still values an 18-decimal bond exactly as before  
refuses to register a 0-decimal token as a bond

UWattIssuer – previewRedeem mirrors the desk  
quotes min(\$1, C) – not raw C – while over-collateralized (350ms)  
agrees with an actual redeem to the wei  
agrees with an actual redeem below par too (328ms)  
nets the desk fee  
fails closed on a stale oracle, exactly like the desk

UWattIssuer – pre-audit regressions (Nethermind scan 3)  
#1 fails closed when the stablecoin feed freezes, on the VALUATION path too  
#1 still prices a stablecoin that is merely OFF PEG, as long as the feed is alive  
#9 stops counting an in-flight T+1 redemption once its settlement window lapses (83ms)  
#8 each T+1 lot expires on its own clock, not the oldest lot's (77ms)  
#8 settle consumes lots FIFO and clears the aggregate (92ms)  
#8 refuses to open more than the bounded number of concurrent lots (138ms)

UWattIssuer – deposit-asset decimals are derived, not trusted (scan 5, F5)  
overrides a mis-configured decimals with the token's real value

UWattIssuer – deviation band bounded by the redemption fee (scan 5, High 1)  
rejects a deposit asset whose band exceeds the fee  
rejects lowering the fee below an already-configured asset's band

196 passing (36s)  
1 pending

## 9 About Nethermind

**Nethermind** is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

**Blockchain Security:** At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

**Blockchain Core Development:** Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

**DevOps and Infrastructure Management:** Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

**Cryptography Research:** At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

**Smart Contract Development & DeFi Research:** Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

**Our suite of L2 tooling:** Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

### General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

### Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.